CHW Navigator v 1

CHW Navigator v1

Created Date: 2026-02-17
Updated Date: 2026-02-17
Key: For the future: highlighted in yellow
Scope: Engineering configuration and verification pipeline for CHW decision-support logic.
Core Philosophy: Clinical content comes from MOH/WHO; Engineering provides the deterministic compiler.

This “Gold Master” is a closed-loop build-and-verify pipeline for turning a CHW clinical manual into an ODK-deployable form whose decision logic is (a) deterministic, (b) missingness-safe, (c) stockout-safe, (d) message-consistent, and (e) tested for equivalence between the DMN reference logic and the runtime form.

At a high level, artifacts fall into four layers:

1. **Instructions layer:**
    a. Prompt_Fragments.csv,
    b. Master_Prompts.csv (The Build definition).
    c. An orchestrator.py that
        i. calls the LLMs with the prompts
        ii. Calls lots of python routines for linting, Z3 proving, translating to XLSForm, etc.
2. **Meaning layer that the prompts extract from the manuals**
    a. FactSheet.csv (Facts + Priority Weights+supply list + etc.)
    b. message_ids.json (Source Text)
    c. internal representation of the logic for the queue of modules (fever, cough, etc.)
3. **Decision tables and flowcharts for clinician review**
    a. symbols.json (Namespace)
    b. predicates.json (turning continuous data into booleans)
    c. Xx.dmn and router.dmn (traffic cops for the queue)
    d. Integrative.dmn. (Integrate care from multiple modules: drug interactions, etc.)
    e. Flowcharts & Shadow_Manual.md (Clinician Sign-off with Flowcharts).
4. **Executable layer that is a module in existing CHW software:**
    a. xlsform_calculates.csv (State Machine).
    b. And similar for FHIR compliant questionnaire
5. **Testing layer:**
    a. CTIO_Standard.json (Test Schema),
    b. synthetic_patients.json (Boundary Triplets, etc.),
    c. z3 prover stuff

Why this approach is credible

We have built a verification pipeline that automates the conversion of MOH guidelines into deployment-ready software (CHT, OpenSRP, CommCare, ODK). Unlike standard human coding, which is prone to typos and omission errors, or standard AI, which can be unpredictable, our system uses a **"Zero-Trust" Engineering Pipeline**.

We ensure the resulting code is safer and more accurate than hand-coded modules through four rigorous layers of defense:

1. Ministry Sovereignty (Human-in-the-Loop) The AI never has the final say. We require human sign-off at every critical stage:

- **Gap Analysis:** Before writing code, the system identifies ambiguities in the manual or guidelines (e.g., missing instructions if a stockout or unclear age limits) and halts for MOH clarification.
- **Logic Review:** Before any code is written, MOH clinicians must approve the clinical decision rules in plain English, in decision tables, and/or as decision flowcharts.
- **Language Review:** MOH clinicians and CHWs verify the exact wording (phrase banks) for local languages.

2. Adversarial "Red Teaming" We do not rely on a single AI model. We use a multi-agent system where two competing AIs draft each step of the process of analyzing the logic. Then a separate, independent "Red Team" AI adjudicates between the two drafts, after which it aggressively hunts for errors.

- If the Red Team finds a flaw, a "Repair" AI fixes it.
- The system loops until the code passes all quality standards. If it cannot satisfy our strict safety criteria, the process halts.

3. Mathematical Proof of Internal consistency: We use formal mathematical verification to prove the logic is internally consistent before we submit for MOH approval. Proofs include:

- every patient path ends in a clear action (Treat, Refer, or Discharge).
- no patient can trigger two conflicting treatments simultaneously.
- Decision tables with only one permissible output are mutually exclusive and collectively exhaustive,
- etc.

4. Massive Patient Simulation We stress-test the software with thousands of synthetic patient profiles to guarantee reliability.

- **Boundary Testing:** We test patients exactly at critical cut-points (e.g., a child exactly 11 months vs. 12 months old) to ensure rules based on continuous measures trigger correctly.
- **Edge Cases:** We simulate complex scenarios, including patients with multiple illnesses, with no symptoms, and with rare combinations, to ensure the guidelines are followed 100% of the time.
- We ensure all simulated cases perform identically when run through the clinical logic approved by MOH and the software running a CHW's phone.

Not spring 2026: More testing

- When “ground truth” decision tables or code exists or we can create it, we tested our system to ensure it can create valid decision tables.
    - When NGOs or Ministries have created hard-coded CHW software based on a manual, we verified our system can generate decision tables that lead to identical diagnoses and treatment.
    - We invent “Martian” manuals of the same complexity and quality as CHW manuals. We verified our system can generate the correct decision tables. As we invented the diseases, we know the correct answers.
    - We hand-coded decision tables for a number of CHW manuals. Medical doctors confirmed our team’s hand-coded tables. We verified our system can generate the correct decision tablets.
- We take the decision tables and have an LLM write a manual based on those tables. We then have a different set of LLMs turn those tables into decision tables and ensure we re-create

The Workflow

This is the workflow from ingesting a manual to creating code that executes as part of existing software on CHWs' phones.

Phase 0 — Governance and setup

1. Define scope: country/program, age bands, care level, target app platform, offline constraints, clinicians at the Ministry of Health (MOH) or NGO who can approve the clinical logic.
2. Establish **source bundle**: manual version + addenda + local policies + WHO's "CCC DAK dictionary" snapshot of many medical terms with related standard codes for diseases, drugs, etc.
3. Define acceptance criteria: required test coverage, sign-off gates

Phase 1 — LLM-assisted extraction (nondeterministic)

4. Ingest manual → chunk + normalize + build citation map
5. Extract:
    - **Fact sheet** (atomic clinical statements) **with provenance** (source document, page #, text snippet)
    - **Symbols table** (IDs, types, units, value sets, synonyms) **with provenance**
    - **Issue log** (“rough edges”of the manual such as not addressing what to do if a drug has stockout) with severity + options + impacted sections

6. QC for each artifact:
    - red team model critique
    - repair loop with max-iterations + deterministic stop rules

i. human escalation for unresolved conflicts

Phase 2 — MOH resolution loop

7. MOH reviews issue log → produces addendum to the manual
8. If an addendum: Repeat Phases 1 to Update symbols & fact sheet with MOH addendum (again provenance-tagged) until no rough edges.

Phase 3 — Generate clinical logic

9. Generate **DMN decision tables** from (facts + resolved addendum)
    a. **Translate phrase bank** to local languages
    b. Note: Updates highlight what is new in the decision table, flowchart, & phrase bank
10. QC:
    - red team + repair loop
    - Schema validation: All messages are in the right syntax
    - DMN & Z3 Linters that check syntax
    - DMN decision tables match their specifications (hit policies like "mutually exclusive" decisions, data typing, rules for missing data, etc.)
    - Phrase bank:
        - QC: A separate AI does back-translation; a third AI adjudicates

---------- DETERMINISTIC BELOW HERE----------

Phase 4 — Formal verification + test synthesis

11. Define invariants catalog that the Z3 prover package will prove (e.g., all decision tables are exhaustive; specified tables are mutually exclusive, no infinite loops, etc.)
12. Compile DMN decision tables → Z3 language for formal proofs
13. Z3 proves the invariants
    a. or releases a memo of problems & halts the process
14. Z3 generate synthetic patients for:
    - each diagnosis and pair of diagnoses
    - boundary thresholds (e.g., 37.4, 37.5 and 37.6°C)
    - conflicting findings
    - Missing data or supplies
    - etc.

Phase 5 — Create flowcharts and show equivalence of Z3, flowchart & DMN

14. Translate DMN → Mermaid flowchart language. Flowcharts are for clinician review only; not source of truth.

15. QC: Run patient suite through DMN, Z3, and Mermaid-derived graph → assert identical outcomes

Phase 6 — Deployment formats + end-to-end validation (deterministic)

17. Translate DMN → XLSForm and/or FHIR Questionnaire
18. Run patient suite through DMN vs XLSForm/FHIR → assert identical outcomes
19. Validate XLSForm + package release bundle (versioned, hashed, signed)

Phase 7 — Approvals + release

20. Submit to MOH: clinical logic (decision tables + flowcharts) + traceability report + test report
21. MOH clinician approval (logic/clinical safety)
    a. Note: This approval could come after phase 5.
22. CHW approval of the phrase bank language and user interface
23. Release with rollback plan. Include monitoring hooks + incident response path

Core pipeline from manual to executable code

Outputs of AI, outputs of translators we write, MOH approval gates

Manual
↓ Orchestrator program with a set of prompts that calls LLMs, Python code to do quality checks and translate code, etc.
↓ Memo on rough edges for the manual
MOH writes clarifying addendum
↓ AI's disassemble the manual into pieces and reassemble them into internal representations: FactSheet, Environmental and context toggles, Symbols table Map of how to turn all decision inputs into true/false boolean variables
↓ **DMN decision tables** with properties such as "decisions are mutually exclusive" **Phrase bank**
------- DETERMINISTIC BELOW HERE--------

↓ Translate DMN into Mermaid flowchart language
↓ Flowchart
MOH approves clinical logic (decision tables & flowcharts) & phrase bank
↓ Translate DMN into XLSForm or FHIR compliant questionnaire
↓ Code runs deterministically on older Android phones as a form within existing CHW software

Including quality control

Core pipeline	Quality control at each step
Manual	
↓ Orchestrator program with a set of prompts that calls LLMs, Python code to do quality checks and translate code, etc.	
↓ Memo on rough edges for the manual	*(LLM red-team + repair loop)*
MOH writes clarifying addendum	
↓ AI's disassemble the manual into pieces and reassemble them into internal representations: FactSheet, Environmental and context toggles, Symbols table Map of how to turn all decision inputs into true/false boolean variables	*(LLM red-team + repair loop)*
↓ **DMN decision tables** with properties such as "decisions are mutually exclusive" **Phrase bank**	*(LLM red-team + repair loop)*
------- DETERMINISTIC BELOW HERE--------	*(LLM red-team + repair loop)*
↓ Translate DMN into Mermaid flowchart language	*Translate DMN into Z3 framework for formal proofs* *Z3 creates simulated patients*

↓ Flowchart	*Test DMN, Z3 and Mermaid have identical results* *Z3 proves no loops, tables are exhaustive, etc.*
MOH approves clinical logic (decision tables & flowcharts) & phrase bank	
↓ Translate DMN into XLSForm or FHIR compliant questionnaire that runs	*Test DMN and XLSForm, etc., have identical results*
↓ Code runs deterministically on older Android phones as a form within existing CHW software	*Processes to report bugs, etc.*

---

1) Artifact Set (Single Source of Truth Design)

Artifact	Produced in	Purpose	Must-have schema checks
Prompt_Fragments.csv	Authoring	Database of pieces of the prompts	Unique IDs; RFC4180 quoting.
Master_Prompts.csv	Authoring	Template that show how to assemble pieces of the prompts into prompts.	All slugs resolvable.
Background_sample_dictionary.csv	Authoring	Standard source for keywords (temp_c, etc.). CCC DAK XLS with a column for CHW Navigator keywords to feed the LLM	Names satisfy naming conventions (from WHO CCC DAK)
Pipeline.yaml	Authoring	Outlines the steps (prompts, quality checks, etc.) to use to step through the pipeline from manual to XLSForm.	YAML linter
Intermediate artifacts the pipeline creates			
FactSheet.csv	B1	Facts + **Priority Weights**	std_factsheet_schema (Provenance required).

Canonical_DataDictionary.csv	B2	Dictionary with **IO Types**	io_type (input/context/output).
message_ids.json	B4	**Source Text** + Placeholders	std_message_schema (Placeholder map).
context_vars.json	B1	Static Toggles (Environmental conditions, supply list)	std_context_schema.
Activators.dmn	TBD	Determine what modules go into the queue	See end of Artifact 2: Prompts
Router.dmn	TBD	Determine what module from the queue to execute next	See end of Artifact 2: Prompts
symbols.json	B_Merge	Master Symbol Table	std_symbols_schema (Namespace + Units).
predicates.json	B5b	Predicate Layer (1/0), turning all decision-relevant data into booleans	std_predicates_schema (Fail-safe).
DMN_Integrative.xml	B7	Integrate care from each module: Drug interactions, etc.	Emergency Exit Rule (Row 1).
xlsform_calculates.csv	B9	ODK Logic	
CTIO_Standard.json	D1	Canonical Test I/O Schema	Unified Input/Output format for patient data
synthetic_patients.json	D1b	Synthetic Test Data	Adheres to Test I/O Schema
golden_cases.yaml	Authored by medical team	Clinician Ground Truth	Adheres to Test I/O Schema
Outputs for approval			
Shadow_Manual.md	D5	Mermaid Flowcharts + Narrative	Clinician approval required.
phrase_bank.csv	F1 (might move earlier, so only one approval round)	Multi-lingual content	std_phrasebank_schema (Status flags). Clinician approval
Output to users			
form_multilingual.xml	F3	Final ODK Form (En+Trans)	Valid XForm standard.

Artifact 2: Prompts

Contains

- Prompt fragments that are re-used. For example:
    - Instructions for red teaming are similar at each phase
    - Quality standards re-appear in many prompts
    - etc.
- Prompt templates, showing how to assemble the fragments in each prompt.

Artifact 3: pipeline_fsm.yaml

This pipeline outlines the steps (prompts, quality checks, etc.) to use to step through the pipeline from manual to XLSForm.

Artifact 4: Sample Schemas

- Placeholders of the the formats for various messages and artifacts
- The team should update as we go through semester,

---

6) Artifact 5: Write-Once Code

These are programs to write once. Green are highest priority.

- Code to run through the pipeline.yaml

- Linters check syntax and add semantics such as keywords are in the symbols table
    - Pipeline.yaml linter to ensure well formed.
    - Namespace: All names and keywords follow our naming rules
    - Mermaid
    - Z3
    - Note: Use an existing DMN linter
    - Most artifacts and schemas:
        - All keywords are in the symbols tables
        - There is a source and a snippet of text for any fact.

- Translators
    - DMN to Mermaid
    - DMN to Z3
    - DMN to XLSForm
- Test harness to run simulated patients through
    - DMN
    - Mermaid graph
    - Z3
    - XLSForm

- Compare clinical outcomes from simulated patients run through different paths

Artifact 6: Red Team critiques by LLMs of this plan

These are a bunch of LLM critiques of the plan.

Naming conventions

Naming conventions

ChatCHW Variable Naming Codebook v1.1 (2026-02-15)

Purpose

A strict, prefix-based naming system for all variables used in manuals → DMN → XLSForm/ODK/CHT/OpenSRP → (optional) FHIR mappings. This codebook enables consistent authoring, automated linting, safe refactoring, and clear provenance (who/what produced a value).

Core principles

1. Provenance first: the prefix signals where the value comes from and how it should be used.

2. Stable meaning: names encode clinical concepts, not full logic.

3. Schema is authoritative: type, allowed values, nullability, and provenance are enforced in the data dictionary/schema.

4. Units are mandatory for numeric values and must use an approved unit token.

5. Computed outputs are never user-entered (enforced by schema + linter).

6. No negations in names: represent negation in values, not identifiers.

7. Missingness is explicit: entered variables may be null; computed booleans must not be null.

Prefix taxonomy

Use lowercase prefixes with an underscore: prefix_concept[_attribute...]

Prefix table

Prefix	Category	Definition	Typical Type(s)	User-entered?	Examples
q_	Question / history	Caregiver/patient-reported responses for this encounter	bool, int, float, enum, string	Yes	q_blood_in_stool, q_cough_duration_days
ex_	Exam / observation	CHW observation (non-“vital”); inspection/palpation/appear	bool, enum, string	Yes	ex_chest_indrawing, ex_feet_swelling_bilateral

		ance			
v_	Vital / measurement	Quantitative measurement (device, count, timed)	int, float	Yes	v_temp_c, v_resp_rate_per_min, v_muac_mm
lab_	Lab / point-of-care test	Diagnostic test result (RDT, dipstick, etc.)	enum, bool, float	Yes	lab_malaria_rdt_result, lab_hb_g_dl
img_	Imaging	Imaging finding/result (if used)	enum, bool	Rare	img_cxr_consolidation
hx_	Baseline history / risk	Chronic/past conditions or stable risks (not visit-specific events)	bool, enum, string	Yes	hx_hiv_exposed, hx_premature_birth
demo_	Demographics	Demographics and identity descriptors (not platform metadata)	int, enum, string, date	Yes	demo_sex, demo_birthdate, demo_caregiver_relation
sys_	System / platform context	Device/app metadata (non-clinical)	string, date, int, float	No	sys_encounter_id, sys_chw_id, sys_gps_lat
prev_	Prior encounter state	Values retrieved from prior encounters in the longitudinal record (previous visit, earlier in episode)	any	No	prev_rx_amoxicillin_end_date, prev_dx_pneumonia
calc_	Calculated	Intermediate	int, float	No	calc_age_mo

	numeric	computed numeric values			nths, calc_weight_for_age_z
p_	Predicate flag	Intermediate computed boolean simplifying tables	bool	No	p_danger_sign_present, p_high_fever
s_	Score	Risk/triage score outputs	int, float	No	s_hypoxemia_risk
dx_	Diagnosis / classification	Computed clinical classifications	bool, enum	No	dx_pneumonia, dx_dehydration_severe
sev_	Severity label	Computed severity/stage label when separated from dx_	enum	No	sev_pneumonia
need_	Workflow need	Activator outputs that drive module routing / queueing	bool	No	need_fever_module, need_followup_3d
tx_	Treatment intent (workflow-significant)	Plan-level actions; may include route/urgency when it changes workflow	bool, enum	No	tx_antibiotic_oral, tx_antibiotic_im, tx_ors
rx_	Medication execution	Dose/frequency/duration/route for a medication	int, float, enum, date	Usually No	rx_amoxicillin_dose_mg, rx_amoxicillin_duration_days
proc_	Procedure	Non-drug procedures	bool, enum	No	proc_oxygen, proc_iv_fluids

ref_	Referral destinations	Referral actions encoded as destination+urgency booleans (not mixed enum/bool)	bool	No	ref_hospital_urgent, ref_nutrition_routine
adv_	Advice / counselling	Caregiver advice outputs	bool	No	adv_return_immediately_if_worse, adv_continue_feeding
fu_	Follow-up scheduling	Follow-up timing and reason fields	int, date, enum, bool	No	fu_in_days, fu_reason
out_	Disposition	Encounter outcome/disposition	enum	No	out_disposition
err_	Data quality / validation	Missingness/ contradiction/ out-of-range flags	bool, enum	No	err_missing_temp, err_inconsistent_age

Naming syntax rules

Allowed characters and case

- Lowercase letters, digits, underscores only.
- No spaces, hyphens, periods, slashes, or camelCase.

Structure

- Required: prefix_concept
- Optional attributes: prefix_concept_attribute_attribute
- Keep names short but specific; do not encode full decision logic in the identifier.

Good / bad examples

Good:

- q_diarrhea_duration_days

- v_resp_rate_per_min
- lab_malaria_rdt_result
- p_danger_sign_present
- dx_pneumonia
- tx_antibiotic_oral
- rx_amoxicillin_dose_mg
- ref_hospital_urgent
- prev_rx_amoxicillin_end_date

Bad:

- q_no_blood_in_stool (negation in name)
- tx_amoxicillin_250mg (strength belongs in rx_)
- ex_temp (missing unit and wrong prefix)
- DiagnosisPneumonia (camelCase, missing prefix)
- q.blood_in_stool (periods)
- v_temp_celsius (unit token must be _c)

Units and numeric conventions (hard requirements)

All numeric variables MUST end with an approved unit token (last token(s)).

Approved unit suffix tokens (expand only by codebook update)

- Temperature: _c, _f
- Length/anthro: _mm, _cm, _m
- Weight: _kg, _g
  Duration: _minutes, _hours, _days, _weeks
- Rates: _per_min, _per_hour
- Dose: _mg, _mcg, _ml, _mg_per_kg
- Concentrations: _g_dl, _mmol_l

Examples:

- v_temp_c
- v_muac_mm
- q_cough_duration_days
- rx_amoxicillin_dose_mg_per_kg
- rx_amoxicillin_duration_days

Boolean conventions

- Booleans represent positive assertions; store false rather than naming “no/not”.
- Prefer q_cough_present only when needed to disambiguate a boolean from a related numeric (e.g., duration).

Enumerations and result coding

Use _result / _status / _reason suffix for multi-valued fields (not for booleans).

- lab_malaria_rdt_result ∈ {positive, negative, invalid, not_done}
- fu_reason ∈ {pneumonia_followup, diarrhea_followup, fever_followup, other}
- out_disposition ∈ {treated_home, referred, emergency_refer, followup_scheduled, other}

Never encode enum options in variable names.

Computed vs entered rules (hard requirements)

Computed-only prefixes (must never appear as user-entered fields)
sys_, prev_, calc_, p_, s_, dx_, sev_, need_, tx_, rx_, proc_, ref_, adv_, fu_, out_, err_
Potentially entered prefixes (may appear on forms)
q_, ex_, v_, lab_, img_, hx_, demo_

Missingness and null-handling (hard requirements)

1. Entered variables are nullable: all q_, ex_, v_, lab_, img_, hx_, demo_ must allow null/blank.
2. Computed booleans are total: all p_, dx_, need_, tx_, ref_, adv_ must be strictly true/false (no null).
3. Computed numerics declare missingness explicitly: if a computed numeric can be

undefined, it must have a paired err_ flag (e.g., calc_age_months + err_missing_birthdate).

4. DMN rules must treat null explicitly: any comparisons with nullable numerics must guard missingness (e.g., q_diarrhea_duration_days null must not silently behave like 0).

Concept boundaries: what goes where

q_ vs hx_ vs prev_

- q_: reported for the current encounter ("this illness now")
- hx_: baseline history/risk that is relatively stable
- prev_: values retrieved from prior encounters (previous visit or earlier episode), not re-asked

Examples:

- prev_rx_amoxicillin_end_date (prior course timing)
- q_taken_any_antibiotics_this_illness (if you choose to ask instead of retrieving)

ex_ vs v_

- v_: numeric measurement with units (including counts like respiratory rate)
- ex_: qualitative/boolean/categorical observation (e.g., chest indrawing)

Treatment modeling: tx_ vs rx_ (workflow-safe rule)

- tx_ is the plan-level action and may include route/urgency when it changes workflow.
- rx_ carries executable details (dose/frequency/duration/route as needed).

Recommended pattern

- tx_antibiotic_oral = true (routine outpatient workflow)
- tx_antibiotic_im = true (pre-referral / injection workflow)
- rx_amoxicillin_dose_mg_per_kg
- rx_amoxicillin_frequency_per_day
- rx_amoxicillin_duration_days
- rx_amoxicillin_route (optional; if route is fully encoded in tx_, route may be redundant in rx_)

Referral modeling: ref_ is boolean-by-destination (hard requirement)

To avoid enum/boolean ambiguity and support multiple simultaneous referrals:

- Use one boolean per (destination × urgency) referral you support.
- Do not use a single enum that can only hold one referral.

Examples:

- ref_hospital_urgent = true
- ref_nutrition_routine = true
- ref_health_facility_routine = true

need_ vs tx_ (Activator vs Router boundary; hard requirement)

- need_ flags are Activator outputs and Router inputs (workflow routing, module queueing, follow-up need).
- tx_/rx_/ref_/adv_ are final clinical outputs for the encounter (patient care instructions).

Reserved tokens (avoid as standalone concepts)

Do not use ambiguous tokens as primary concepts:
data, value, flag, thing, misc, temp (unless temperature), test (unless a specific lab), other (unless enum option), status
Examples to avoid:

- tx_status (use out_disposition or a specific boolean)
- q_value (use a real concept name)

Versioning and deprecation

- Never silently rename a variable.
- If renaming: add an alias map old_name → new_name and keep both during migration.
- Deprecation is recorded in schema metadata, not in the name (lifecycle = deprecated, replaced_by = ...).

Lint specification

Primary regex (syntax)

^(q|ex|v|lab|img|hx|demo|sys|prev|calc|p|s|dx|sev|need|tx|rx|proc|ref|adv|fu|out|err)_[a-z][a-z0-9]*(?:_[a-z0-9]+)*$

Semantic lint rules (must be checked against the data dictionary)

1. Unit enforcement: if type ∈ {int,float} then name must end with an approved unit token.

2. Computed-only enforcement: computed-only prefixes cannot bind to user input widgets.

3. No negations: reject names containing _no_, _not_, _without_.

4. No strength-in-tx_: forbid dosage/strength tokens in tx_ names; require rx_ fields instead.

5. Uniqueness: variable names are globally unique per project version.

6. Type stability: a variable name cannot change declared type across versions without explicit migration.

7. Null discipline: computed booleans must be total; nullable inputs must be guarded in DMN comparisons.

8. Referral discipline: ref_ variables must be boolean and encode destination+urgency.

Minimal required fields for every variable in the Data Dictionary

Every variable must have:

- name
- prefix_category (inferred)
- type (bool/int/float/enum/string/date/datetime)
- units (required if numeric; null otherwise)
- nullable (true/false)
- allowed_values (required if enum)
- entered_by (caregiver/chw/system/computed)
- module (e.g., fever, diarrhea, cough)
- question_text (for q_/ex_/v_/lab_ where applicable; per language)
- source_guideline_reference (page/section/citation pointer)
- notes (measurement method, edge cases, etc.)

Quick examples (IMCI-style diarrhea + pneumonia)

- q_diarrhea_duration_days (int, units=days, nullable=true)

- q_blood_in_stool (bool, nullable=true)
- v_temp_c (float, units=c, nullable=true)
- v_resp_rate_per_min (int, units=per_min, nullable=true)
- ex_chest_indrawing (bool, nullable=true)
- p_danger_sign_present (bool, nullable=false, computed)
- dx_pneumonia (bool, nullable=false, computed)
- dx_dehydration_severe (bool, nullable=false, computed)
- need_followup_3d (bool, nullable=false, computed)
- tx_antibiotic_oral (bool, nullable=false, computed)
- rx_amoxicillin_dose_mg_per_kg (float, units=mg_per_kg, nullable=true/false per design + err_ flag)
- rx_amoxicillin_duration_days (int, units=days, nullable=false)
- ref_hospital_urgent (bool, nullable=false, computed)
- fu_in_days (int, units=days, nullable=false)

Change control

This codebook is a clinical safety artifact. Updates require:

1. written rationale + examples
2. updated lint rules if needed
3. migration notes (aliases) if names change
4. clinical review sign-off if changes touch dx_, tx_, rx_, ref_, or need_ namespace

NOTE: We want DMN to use simple Booleans. But compressing to Booleans hides a lot of information.

- When creating a derived variable such as “fast breathing = RR <60/min. & 12 <=age < 59 mo. Or RR < 50 and age < 12 mo.”
- Keep the human readable text as the variable label
- In the mermaid, show construction of variable

Prompts

Prompts

Note that the prompts need to work on the "traffic cop" DMN tables; see Clarifying module flow below.

2) Artifact 1: Prompt_Fragments.csv

Code snippet

None

```
Fragment_ID,Category,Content
header_yaml,Metadata,"artifact_id: {{id}}
canonical_version: v5.1
merge_policy: strict_append
owners: System_Admin
timestamp: {{timestamp}}"

role_redteam,Role,"You are an adversarial Clinical QA
Auditor (Red Team).
YOUR GOAL: Break the system. Find ""silent
killers""—logic gaps that don't crash the compiler but
kill the patient.
MINDSET:
1. Distrust everything. If the manual says ""Give
Amoxicillin,"" ask ""What if the clinic is out of
stock?""
2. Hunt for Ambiguity. ""Child is hot"" is a failure.
""Temp > 37.5"" is a pass.
3. Attack the State Machine. Can I get stuck in a loop?
Can I skip a critical step?
OUTPUT: A ruthless list of vulnerabilities, ranked by
clinical severity."
```

```
role_clinical_auditor,Role,"You are a Clinical Protocol
Analyst and Public Health Expert (specializing in WHO
IMCI).
YOUR GOAL: Ensure the engineering artifacts faithfully
represent the clinical intent.
MINDSET:
1. The Patient comes first.
2. The Manual is the law, but the Manual is often
flawed. Identify contradictions.
3. Clarity is safety. If a logic flow is confusing to
you, it will be deadly to a CHW.
OUTPUT: Clear, plain-English narratives and specific
queries for the Ministry of Health."

role_knowledge_engineer,Role,"You are a Senior
Knowledge Engineer.
YOUR GOAL: Extract structured truth from unstructured
prose.
MINDSET:
1. Atomic Facts. Break complex sentences into single,
testable assertions.
2. Separation of Concerns. Clinical facts (symptoms) !=
Program logic (referral rules).
3. Hierarchy. Identify the ""Triage Order""—what must
be checked first?
OUTPUT: CSVs and JSONs that capture the *structure* of
the clinical domain."

role_data_modeler,Role,"You are a Clinical Data
Modeler.
YOUR GOAL: Define the rigid schema that governs the
system.
MINDSET:
1. Namespaces are non-negotiable. You explicitly reject
""naked"" variable names.
```

```
2. Types are strict. A boolean is 1/0, never
""true""/""false"".
3. IO is explicit. You distinguish between an
Observation (Input) and a Decision (Output).
OUTPUT: A Canonical Dictionary and Symbol Table that
compiles without warnings."

role_dmn_architect,Role,"You are a DMN 1.5 Logic
Architect.
YOUR GOAL: Decompose prose into deterministic Decision
Tables.
MINDSET:
1. MECE (Mutually Exclusive, Collectively Exhaustive).
Every row has a unique hit.
2. No Math in Logic. Math happens in predicates; DMN
only evaluates 1/0 flags.
3. Stateless. DMN does not remember the past; it only
evaluates the current inputs.
OUTPUT: Valid DMN XML that adheres to the
""Antigravity"" subset."

role_compiler,Role,"You are an XLSForm Engineer (ODK
Specialist).
YOUR GOAL: Turn abstract logic into a runnable,
crash-proof mobile form.
MINDSET:
1. The Phone is hostile. It may be offline, low
battery, or have a broken screen.
2. Users are unpredictable. They will skip questions or
enter garbage data.
3. Logic must be explicit. Never rely on implicit
casting.
OUTPUT: XLSForm logic that manages state (bitmasks) and
routing explicitly."
```

```
std_logic_integrity,Standard,"STANDARD: Decision Logic
Integrity
1. MECE Rule: Every decision table must cover 100% of
the input space. Use an explicit ""ELSE"" row for every
table.
2. Termination Rule: Every logic path must end in a
valid Disposition (REFER, TREAT, OBSERVE, HALT). No
dead ends.
3. Monotonicity Rule: Adding a ""Danger Sign"" input
must NEVER downgrade the output (e.g., from Refer to
Treat).
4. No Phantom Inputs: Every input variable used in
logic must appear in the Data Dictionary."

std_safe_endpoints,Standard,"STANDARD: Safety &
Endpoints
1. Safe Default: If logic fails or data is missing, the
system must default to the HIGHER risk category.
2. Fail-Safe Values: Every check (e.g., ""Is Fever
High?"") must return '1' (True) if the data is
missing/unknown, until proven otherwise.
3. Emergency Exit: The ""Urgent Referral"" path must be
a ""Short Circuit"". If triggered, no further clinical
questions are asked."

std_missingness_model,Standard,"STANDARD: Missingness
Model (ODK-Safe)
1. THE BOOLEAN TRAP: In ODK/XPath,
string-length('false') is > 0, so boolean('false')
evaluates to TRUE.
   - RULE: Never use strings 'true'/'false'. Use
numeric 1 and 0.
2. Explicit Presence:
   - Define: p_present := string-length(${v_raw}) > 0
```

```
   - Define: p_value := if(p_present, ${v_raw},
fail_safe_value)
3. Fail-Safe:
   - If a thermometer is broken (missing), p_is_fever
MUST default to 1 (High Risk) to trigger the referral."

std_antigravity_data,Standard,"STANDARD: Antigravity
Namespacing (Strict Prefixing)
You must enforce a Hungarian-style naming convention to
prevent compiler errors.
1. v_[slug]: Raw Value / Observation. (e.g., v_temp_c,
v_cough_duration). Input from user/sensor.
2. p_[slug]: Predicate / Derived Logic. (e.g.,
p_is_fever_high, p_has_pneumonia). ALWAYS 1/0.
3. c_[slug]: Context Variable. (e.g., c_malaria_zone,
c_stock_amox). Environment settings.
4. m_[slug]: Message ID. (e.g., m_tell_give_amox).
Output strings.
VIOLATION: Any variable without a prefix (e.g.,
""temp"", ""fever"") is a CRITICAL ERROR."

std_queue_management,Standard,"STANDARD: Agnostic Queue
& Priority Logic
1. The Emergency Interrupt:
   - IF (p_danger_sign == 1 OR p_unconscious == 1)
   - THEN next_module = 'END_REFERRAL'
   - EFFECT: Bypasses all other queues.
2. The Priority Lattice:
   - Modules are ordered by 'priority_weight'
(extracted from FactSheet).
   - Lower number = Higher priority (1 is first).
3. State Awareness (The Bitmask):
   - A module is only valid if: (p_req_[module] == 1)
AND (p_[module]_complete == 0).
4. Acyclic Guarantee:
```

```
   - Modules cannot re-trigger themselves. The queue
moves forward or terminates."

std_queue_syntax,Standard,"STANDARD: Queue IR Syntax
(JSON)
The LLM must output the Queue logic in this exact JSON
schema:
{
  ""emergency_triggers"": [""p_danger_signs"",
""p_convulsions""],
  ""priority_map"": [
    { ""rank"": 1, ""module_id"": ""respiratory"",
""trigger_predicate"": ""p_check_cough"" },
    { ""rank"": 2, ""module_id"": ""diarrhea"",
""trigger_predicate"": ""p_check_stool"" }
  ],
  ""termination_condition"": ""END_SESSION""
}"

std_message_contract,Standard,"STANDARD: Message
Contract & Placeholders
1. Single Source of Truth: English text lives ONLY in
message_ids.json. DMN outputs IDs (m_slug).
2. Placeholder Syntax:
   - Text: ""Give {dose} tablets of {drug}.""
   - Map: { ""{dose}"": ""${v_calc_dose}"", ""{drug}"":
""${c_drug_name}"" }
3. Translation Safety:
   - Translators must preserve the {token} exactly.
   - Values must be computed in logic (predicates), not
in the text string."

std_dmn_subset,Standard,"STANDARD: DMN Antigravity
Subset
To ensure compilation to ODK, DMN must be restricted:
```

```
1. Hit Policy: FIRST (Priority) or UNIQUE only. No
COLLECT.
2. Inputs: Only p_ (predicates) and c_ (context). NO v_
(raw values).
3. Cells: boolean (true/false) or ""-"" (Any). No FEEL
expressions (e.g., ""> 38.5"").
4. Math Separation: All threshold logic (> < =) must
happen in the Predicate Layer, not the DMN."

std_odk_naming,Standard,"STANDARD: ODK Naming
Constraints
1. No Spaces: Use underscores.
2. Lowercase: All variable names must be lowercase.
3. No Special Chars: Only alphanumeric and underscores.
4. Mapping: Canonical IDs (patient.vitals.temp) must
map to ODK names (v_temp_c) in symbols.json."

std_test_rubric_dmn,Standard,"STANDARD: DMN Test Rubric
1. Boundary Analysis: For every threshold (X), test
X-0.1, X, and X+0.1.
2. Combinatorial Coverage: Every rule in the DMN must
be triggered by at least one test case.
3. Missingness Suite: Every required input must be
tested as ""NULL"" to verify the fail-safe triggers."

std_ctio_schema,Standard,"STANDARD: Canonical Test I/O
(CTIO)
All test data must follow this schema for differential
testing:
{
  ""case_id"": ""string"",
  ""inputs"": {
    ""observations"": { ""v_slug"": value },
    ""predicates"": { ""p_slug"": 0/1 },
    ""context"": { ""c_slug"": 0/1 }
```

```
  },
  ""expected_outcomes"": {
    ""disposition"": ""REFER|TREAT|OBSERVE"",
    ""messages"": [""m_slug""]
  }
}"

footer_safety_general,Safety,"
### SAFETY HALT CHECKLIST
Before generating output, ask yourself:
1. Did I assume any clinical knowledge not in the text?
(If yes, STOP).
2. Are there ambiguous terms? (If yes, output
ACTION='HALT').
3. Did I follow the Namespace Syntax (p_/v_/c_)?
"

footer_safety_dmn,Safety,"
### LOGIC SAFETY CHECKLIST
1. Does every table have an ELSE row?
2. Are all inputs strictly Predicates (p_) or Context
(c_)?
3. Did I avoid all raw math in the cells?
4. Is the Priority Order respected?
"
```

---

3) Artifact 2: Master_Prompts.csv (Full Fidelity)

Code snippet

None

```
Phase,Prompt_ID,Input_Artifacts,Output_Artifacts,The_Pr
ompt_Template
```

```
A,A1_make,"Manual.pdf","Draft_Addendum.md,
Queries.md","{{header_yaml}}
{{role_clinical_auditor}}

# OBJECTIVE
Analyze the source manual to identify clinical logic
gaps, contradictions, and ambiguities that must be
resolved before engineering begins.

# STANDARDS
{{std_data_quality}}
{{std_supplies_ops}}

# INSTRUCTIONS
1. **Read & Extract:** Read the manual. Identify every
decision point (e.g., ""If fever, do X"").
2. **Identify Gaps:** Look for:
   - Missing dosages or age bands.
   - Contradictory instructions (Page 10 vs Page 40).
   - Ambiguous terms (e.g., ""fast breathing"" without
a rate).
3. **Produce Queries:** For every gap, write a specific
question for the Ministry of Health.
4. **Draft Addendum:** Propose a rigid rule for each
gap to act as a placeholder until the MOH responds.

{{footer_safety_general}}"

A,A2_make,"Manual.pdf,
final_addendum.md","manual_redteam_report.md","{{header
_yaml}}
{{role_redteam}}

# OBJECTIVE
```

```
Perform a hostile audit of the finalized manual and
addendum. Find ""Silent Killers.""

# STANDARDS
{{std_safe_endpoints}}
{{std_missingness_model}}

# INSTRUCTIONS
1. **Stockout Attack:** What happens if the clinic has
0 stock of the primary drug? Is there a fallback?
2. **Missing Data Attack:** If a CHW cannot measure
temperature, does the protocol default to ""High
Risk""?
3. **Loop Attack:** Are there any circular references
(A -> B -> A)?
2. OUTPUT: A valid JSON list of issues: [{ ""issue"":
""..."", ""severity"": ""critical"" }]

{{footer_safety_general}}"

A,A2_repair,"manual_redteam_report.md,
Manual.pdf","manual_draft_repairs.md","{{header_yaml}}
{{role_clinical_auditor}}

# OBJECTIVE
Propose specific, verbatim text amendments to fix the
findings of the Red Team.

# STANDARDS
{{std_logic_integrity}}

# INSTRUCTIONS
1. **Review Findings:** Analyze the
'manual_redteam_report.md'.
```

```
2. **Draft Fixes:** For every ""High Severity"" issue,
write a specific text override.
   - Example: Change ""Give Amoxicillin"" to ""Give
Amoxicillin. If out of stock, refer to hospital.""
3. **Constraint:** Do not invent clinical protocols. If
a fallback is unknown, the fix must be ""Refer"".

{{footer_safety_general}}"

B,B1_make,"Manual.pdf,
final_addendum.md","FactSheet.csv,
context_vars.json","{{header_yaml}}
{{role_knowledge_engineer}}

# OBJECTIVE
Extract atomic clinical facts and the Master Priority
Lattice.

# STANDARDS
{{std_data_quality}}
{{std_antigravity_data}}

# INSTRUCTIONS
1. **Atomic Extraction:** Extract every clinical
condition and action into 'FactSheet.csv'.
2. **Context Variables:** Extract program settings
(e.g., malaria_zone) into 'context_vars.json'.
3. **TREASURE - Priority Extraction:**
   - Analyze the Table of Contents or Triage Section.
   - Assign a 'priority_weight' (integer) to every
Clinical Module.
   - Rule: 1 = Highest (Danger Signs). 2 =
Emergency/Resuscitation. 3+ = Routine assessments.
   - Output this weight in a new column
'priority_weight' in FactSheet.csv.
```

```
4. **Provenance:** Every row must include a 'Source'
column (Manual Page/Para).

{{footer_safety_general}}"

B,B2_make,"FactSheet.csv,
WHO_DAK.csv","Canonical_DataDictionary.csv","{{header_y
aml}}
{{role_data_modeler}}

# OBJECTIVE
Define the rigid Data Dictionary and enforce Namespace
Syntax.

# STANDARDS
{{std_who_strict}}
{{std_antigravity_data}}

# INSTRUCTIONS
1. **Map to Standards:** Map local terms to WHO DAK
codes where possible.
2. **TREASURE - Namespace Enforcement:**
   - Assign an 'odk_name' to every variable using the
strict prefixes:
   - v_[slug]: Input/Observation (raw value).
   - p_[slug]: Predicate (calculated boolean).
   - c_[slug]: Context (env setting).
   - m_[slug]: Message (output text).
3. **IO Typing:** Add a column 'io_type'
(input/context/output).

{{footer_safety_general}}"

B,B4_make,"FactSheet.csv,
Manual.pdf","message_ids.json","{{header_yaml}}
```

```
{{role_knowledge_engineer}}

# OBJECTIVE
Define the Message Architecture (Source Text +
Placeholders).

# STANDARDS
{{std_message_contract}}
{{std_schema_validation}}

# INSTRUCTIONS
1. **Extraction:** For every 'm_[slug]', extract the
exact English Source Text from the manual.
2. **Placeholders:** Identify all dynamic values (e.g.,
{dose}, {days}).
3. **TREASURE - Placeholder Map:**
   - For every placeholder, define the variable source.
   - Format: ""{dose}"": ""${v_calculated_dose}""
4. **Output:** A valid JSON registry.

{{footer_safety_general}}"

B,B5b_make,"FactSheet.csv, context_vars.json,
symbols.json","predicates.json","{{header_yaml}}
{{role_data_modeler}}

# OBJECTIVE
Create the Predicate Layer (1/0 Logic) with Missingness
Protection.

# STANDARDS
{{std_missingness_model}}
{{std_antigravity_data}}

# INSTRUCTIONS
```

```
1. **Define Predicates (p_):** For every clinical
decision point, create a 'p_[slug]'.
2. **TREASURE - The Boolean Trap:**
   - Ensure the XPath expression returns number 1 or 0.
NEVER 'true'/'false'.
3. **TREASURE - Fail Safe:**
   - Define 'fail_safe_value' for every predicate.
   - If 'v_temp' is missing, 'p_fever' must default to
1 (High Risk).
   - Define 'present_test' XPath:
string-length(${v_temp}) > 0.
4. **Dependency Check:** Only use variables present in
'symbols.json'.

{{footer_safety_dmn}}"

B,B6_make,"symbols.json","DMN_Modules.xml","{{header_ya
ml}}
{{role_dmn_architect}}

# OBJECTIVE
Decompose logic into Boolean-Only DMN Tables.

# STANDARDS
{{std_dmn_subset}}
{{std_logic_integrity}}

# INSTRUCTIONS
1. **Inputs:** Use ONLY 'p_' predicates and 'c_'
context vars. NO 'v_' raw values.
2. **Outputs:** Use ONLY 'm_' message IDs and codified
actions (REFER, TREAT).
3. **Structure:**
   - Hit Policy: FIRST.
   - Every table must have an ELSE row.
```

```
   - No FEEL math in cells (all math happens in B5b).

{{footer_safety_dmn}}"

B,B7_make,"DMN_Modules.xml, FactSheet.csv,
symbols.json","DMN_Integrative.xml,
Priority_Lattice.json","{{header_yaml}}
{{role_dmn_architect}}

# OBJECTIVE
Create the 'Master Traffic Controller' (Integrative
DMN) and Queue Definition.

# STANDARDS
{{std_queue_management}}
{{std_queue_syntax}}

# INSTRUCTIONS
1. **TREASURE - Emergency Exit:**
   - Create the Top-Level Rule (Row 1).
   - IF (p_danger_signs == 1 OR p_unconscious == 1)
THEN Next_Module = 'END_REFERRAL'.
2. **TREASURE - Priority Lattice:**
   - Order the remaining rules based on
'priority_weight' from FactSheet.csv.
   - Lower weight = Higher position in table.
3. **TREASURE - State Awareness:**
   - Add an input column 'p_[module]_complete'.
   - Rule: IF (p_req_[module] == 1 AND
p_[module]_complete == 0) THEN Next_Module =
'[module]'.
4. **Output:** The DMN XML and the
'Priority_Lattice.json' (IR).

{{footer_safety_dmn}}"
```

```
B,B9_make,"DMN_Integrative.xml, Priority_Lattice.json,
symbols.json","xlsform_calculates.csv","{{header_yaml}}
{{role_compiler}}

# OBJECTIVE
Compile the ODK State Machine (Calculates).

# STANDARDS
{{std_logic_integrity}}
{{std_antigravity_data}}

# INSTRUCTIONS
1. **TREASURE - State Persistence:**
   - Implement the bitmask logic.
   - Create a calculate: 'completed_bits' that
concatenates module slugs.
   - Map 'p_[module]_complete' to:
if(contains${completed_bits}, 'module'), 1, 0).
2. **DMN Compilation:**
   - Translate the DMN Integrative table into a
cascading if() calculation for 'next_module'.
3. **Formatting:**
   - Ensure all output syntax is valid ODK XPath.

{{footer_safety_dmn}}"

C,C1_make,"FactSheet.csv, message_ids.json,
symbols.json","xlsform_survey.csv","{{header_yaml}}
{{role_compiler}}

# OBJECTIVE
Build the UI Skeleton and Message Registry.

# STANDARDS
```

```
{{std_message_contract}}
{{std_odk_naming}}

# INSTRUCTIONS
1. **Question Logic:** Build the survey. Use 'v_[slug]'
for names.
2. **Message Registry:**
   - Create a hidden select_one 'messages'.
   - Use 'm_[slug]' as choice names.
   - Use 'jr:choice-name' to retrieve text dynamically.
3. **Placeholders:**
   - Ensure the placeholder mapping logic (from B4) is
applied to the output.

{{footer_safety_general}}"

D,D1b_make,"predicates.json,
symbols.json","synthetic_patients.json","{{header_yaml}
}
{{role_qa_engineer}}

# OBJECTIVE
Generate Synthetic Data for Boundary Testing.

# STANDARDS
{{std_test_rubric_dmn}}
{{std_ctio_schema}}

# INSTRUCTIONS
1. **TREASURE - CTIO Schema:**
   - All output must strictly follow the
'CTIO_Standard.json' schema.
   - Use 'v_[slug]' for observations.
2. **Boundary Triplets:**
```

```
   - For every numeric threshold X (e.g. 37.5),
generate three patients:
   - Patient A: X - 0.1
   - Patient B: X
   - Patient C: X + 0.1
3. **Missingness:** Generate cases with 'null' values
for every required input.

{{footer_safety_general}}"

D,D5_make,"DMN_Integrative.xml, message_ids.json,
Priority_Lattice.json","Shadow_Manual.md","{{header_yam
l}}
{{role_clinical_auditor}}

# OBJECTIVE
Create the Human-Readable Logic Mirror for Sign-Off.

# STANDARDS
{{std_logic_integrity}}

# INSTRUCTIONS
1. **Executive Summary:** List major logic changes
since last version.
2. **TREASURE - Queue Narrative:**
   - Analyze 'Priority_Lattice.json'.
   - Write a plain English explanation of the Triage
Order.
   - Explicitly list the ""Emergency Exit"" triggers
(The Red Line).
3. **TREASURE - Mermaid Flowchart:**
   - Generate Mermaid.js code visualizing the State
Machine.
   - Highlight the 'Referral' path in RED.
```

```
4. **Module Logic:** Describe every DMN table as ""If
[Condition] Then [Action]"" prose.

{{footer_safety_general}}"

F,F1_make,"message_ids.json","phrase_bank_en.csv","{{he
ader_yaml}}
{{role_translator}}

# OBJECTIVE
Generate the Master Phrase Bank for Translation.

# STANDARDS
{{std_message_contract}}

# INSTRUCTIONS
1. **Flatten Structure:** Convert message_ids.json into
CSV.
2. **Columns:** message_id, language_code (en), text,
status (verified).
3. **Constraint:** Do not modify placeholders.

{{footer_safety_general}}"

G,G1_make,"[All Logs]","Manifest.json,
Governance_Log.md","{{header_yaml}}
{{role_governance}}

# OBJECTIVE
Final Deployment Manifest and Hash Log.

# STANDARDS
{{std_schema_validation}}

# INSTRUCTIONS
```

```
1. **Hashing:** Calculate SHA-256 hashes for every
Artifact and every Rendered Prompt.
2. **Audit:** List all 'Manual Repairs' (A2) and
'Clinician Sign-offs' (D5).
3. **Manifest:** Emit the final JSON manifest for the
deployment server.

{{footer_safety_phi}}"
```

A new version of b1 to make module flow tables melding 2 formats, sets of inputs, etc.

- But this version does not create the named DMN tables described below.

```
B,B1_make,"Manual.pdf, final_addendum.md","FactSheet.csv,
context_vars.json, Clinical_Trigger_Map.csv","{{header_yaml}}
{{role_knowledge_engineer}}

# OBJECTIVE
Extract atomic clinical facts, Context Variables, and the detailed
Clinical Trigger Map.

(Clinical_Trigger_Map + Triage_Lattice)
Version: 2026-02-04
Role: You are a careful clinical-workflow extractor. Your job is to
read a CHW manual excerpt and produce two CSV-style tables that
define (1) how modules are triggered and (2) how modules are
prioritized/routed. You must NOT invent clinical content. If the
manual is silent, leave a clear placeholder in Notes_for_MD.

INPUTS I WILL PROVIDE
A) Manual excerpt text (may be messy, out of order, with sidebars).
B) Variable dictionary (preferred: <<refer to our source dictionary
xx.csv>>; otherwise, create placeholders v_* with short definitions).
C) List of candidate modules (optional; if absent, infer modules from
manual headings and chief complaints).

OUTPUTS YOU MUST PRODUCE (in this order)

1) DMN table to put modules on queue
2) DMN table to pick next module in queue
```

3) “Assumptions & Ambiguities” list (bullets, each tagged: [FATAL] [MAJOR] [MINOR])
4) “Questions for MD review” list (bullets)

GENERAL RULES
- Do not add modules or triggers not supported by the manual excerpt. If you think one is implied, include it only as a placeholder row with Trigger_Expression = [INSERT] and explain in Notes_for_MD.
- Every Trigger_Expression must be a single boolean expression. No prose in the expression field.
- Timing must be explicit for every trigger: choose one of
  ON_INTAKE_SUBMIT | AFTER_MEASUREMENTS | ON_MODULE_EXIT(<Module>) | CONTINUOUS
- Determinism: The queue must have unique ordering. If two rows share Priority_Rank, you MUST set TieBreak to make the order deterministic.
- Interrupt semantics: any “urgent referral / emergency” behavior must specify Preempt_Mode:
  CLEAR_QUEUE | PAUSE_QUEUE | SOFT_INTERRUPT
- Duplicate handling: every trigger row must specify Duplicate_Policy:
  SKIP_IF_IN_QUEUE_OR_COMPLETED | SKIP_IF_IN_QUEUE | ALLOW_RERUN_IF_COMPLETED
- If the manual gives thresholds (e.g., MUAC cutoffs), copy them exactly; do not “correct” them.

STEP-BY-STEP METHOD (YOU MUST FOLLOW)
Step 1: Identify modules
- List all modules explicitly named (e.g., Cough/Pneumonia, Diarrhea, Fever, Malnutrition, Danger Signs, Referral).
- If headings are absent, infer modules from repeated problem areas; note inference in Assumptions.

Step 2: Extract triggers (Clinical_Trigger_Map)
Create one row per distinct trigger pathway:
- Chief complaint selections / presenting problems
- Universal screening measures done for everyone
- Cross-module triggers (finding in one module enqueues another)
- System triggers (queue empty => EndSession)
- Urgent referral

For each trigger, populate:
Trigger_ID: TRG_### sequential
Target_Module
Trigger_Type: ChiefComplaint | UniversalScreen | CrossModule | System
Source_Location: quote the manual section title and (if available) page number
Eval_Timing: from allowed set
Trigger_Event_Description: 5–20 words, clinician-readable
Evidence_Variables: comma-separated list of variables used
Trigger_Expression: boolean expression using those variables
Action: ENQUEUE(<Module>) | INTERRUPT(Referral) | TERMINATE

Duplicate_Policy: from allowed set (N/A allowed for TERMINATE/INTERRUPT)
Notes_for_MD: short, includes any uncertainty and variable definitions if needed

Step 3: Extract routing/priority rules (Triage_Lattice)
Create one row per routing rule that determines next steps:
- Emergency/danger sign interrupts first
- Then queued modules in priority order
- Then termination when queue empty

Populate:
Rule_ID: TRI_### sequential
Priority_Rank: integer (0 highest, 99 end)
TieBreak: integer (0,1,2...) to break ties deterministically
Module_or_State: module name OR Referral OR EndSession
Applies_When: usually ALWAYS, unless manual scopes by age/setting
Trigger_Condition: boolean expression that references module_triggered('<Module>') or clinical variables
Behavior: INTERRUPT | QUEUE_IF_NEW | TERMINATE | RUN_IF_TRIGGERED
Preempt_Mode: CLEAR_QUEUE | PAUSE_QUEUE | SOFT_INTERRUPT | NONE
Rerun_Policy: NO_RERUN_IF_COMPLETED | ALLOW_RERUN_IF(condition) | N/A
PostAction: RUN_MODULE(<Module>) | GOTO(Referral) | END | [INSERT]
Notes_for_MD: short

DEFINITIONS YOU MAY USE (ONLY THESE)
- module_triggered('<Module>') is TRUE if any trigger row enqueued that module (in Trigger_Map) or set its trigger flag.
- queue_size is the current size of the queue.
- completed('<Module>') is TRUE if module is in completed list.

REQUIRED OUTPUT TABLE SCHEMAS
Clinical_Trigger_Map.csv columns (exactly):
| Trigger_ID | Target_Module | Trigger_Type | Source_Location | Eval_Timing | Trigger_Event_Description | Evidence_Variables | Trigger_Expression | Action | Duplicate_Policy | Notes_for_MD |

Triage_Lattice.csv columns (exactly):
| Rule_ID | Priority_Rank | TieBreak | Module_or_State | Applies_When | Trigger_Condition | Behavior | Preempt_Mode | Rerun_Policy | PostAction | Notes_for_MD |

QUALITY CHECKS (YOU MUST RUN AND REPORT)
QC1: Coverage — every module named in Triage_Lattice (except EndSession/Referral) must be triggerable via at least one Trigger_Map row. If not, add a placeholder trigger row and mark [FATAL].
QC2: Determinism — for each Priority_Rank, TieBreak values must be unique. If not, fix.
QC3: Timing validity — every Trigger_Expression must only use variables available by its Eval_Timing (flag any violation [MAJOR]).
QC4: Interrupt clarity — every INTERRUPT row must specify Preempt_Mode and what happens next (PostAction).

```
# STANDARDS
{{std_data_quality}}
{{std_antigravity_data}}


{{footer_safety_general}}"
```

Clarifying module flow

Module flow

- The internal representation of module flow has at least some of these elements:
    - initial question(s) to determine "What brings you here today?"
        - choices like fever/ cough / diarrhea
    - Initial screening questions asked of all patients /caregivers (or measures such as MUAC tape for malnutrition)
    - Modules can add modules to the queue: "If you find a fever in the diarrhea module => add fever module to the queue (unless already done)"
    - Some manuals may have an "emergency referral to hospital empties all or most of the queue"
    - When done with a module, put it in the “completed module” list
- The format has to be
    - Ideally DMN
    - In any case
        - Close to what MDs review AND
        - Easy for Python to compile into XLSForm or FHIR Questionnaire.

This is the formal specification for the Two-Table DMN Architecture.
It replaces your old CSVs with two standard Decision Model and Notation (DMN) tables. This separation of concerns—Clinical Detection vs. Workflow Routing—makes the logic easier for interns to write and for doctors to audit.

Table 1: The Activator (Activator.dmn)
Role: The "Clinical Detective." It looks at the patient's data (symptoms, vitals) and flags every potential problem or need. It does not care about order, priority, or if the module was already done.
* Hit Policy: COLLECT (Find all valid rules).

* Input: Raw Facts (v_temp, v_cough, v_muac).
* LLM Prompts need to gather all facts that affect module flow and make them columns here.
** Probably they will be booleans before they become columns. So code will turn raw data into those booleans before coming to this table (if the raw data exists). I am not sure how to create the right booleans at the right time.
* Output: A list of strings (required_modules).

Schema Definition

Column Type	Variable Name	Description	Data Type
Input 1	v_symptoms_list	List of selected symptoms from Intake.	list<string>
Input 2	v_temp_c	Measured temperature in Celsius.	number
Input 3	v_muac_color	Mid-Upper Arm Circumference color code.	string ("Red", "Yellow", "Green")
Input 4	v_age_months	Patient age in months.	number
...	...other inputs...	Add columns as needed for new triggers.	-
Output	module_id	The ID of the module to trigger.	string

Intern Instructions: How to Write Activator Rules

Goal: Translate the manual's "Entry Criteria" into a row.

* Find the Trigger: Look for phrases like "Children with cough or difficult breathing should be assessed for pneumonia" or "If MUAC is Red, immediately refer."
* Define the Input Conditions:
  * If the manual says "Fever > 37.5," write > 37.5 in the v_temp_c column.
  * If the manual says "Cough reported," write list contains(?, "cough") in the v_symptoms_list column.
  * Crucial: Leave unrelated columns blank (dash -). This means "doesn't matter."
* Define the Output: Write the standard module ID (e.g., mod_pneumonia) in the Output column.
* One Rule Per Trigger: If a module has two ways to start (e.g., mother reports fever OR thermometer measures fever), write two separate rows. The COLLECT policy will handle the "OR" logic automatically.

Note: De-duplicate modules after this DMN outputs a list of modules to possibly call.
Note: If Activator ever outputs a flag, not a module, then slightly different notation may be helpful.

Table 2: The Router (Router.dmn)

Role: The "Traffic Cop." It looks at the list of potentially needed modules (from the Activator) and the list of completed modules to decide the single next step.

* Hit Policy: FIRST (Priority Order). The engine scans from Top to Bottom and stops at the first match. (or do modules have priorities, so scans the list?)
* Input: required_modules (List from Activator), completed_modules (List from App State).
* Output: next_activity (String). Module name (or maybe also flags for referral)

2027 Note: Sometimes it is ok to redo a module. So we need to have that permission as well.

Schema Definition

Column Type	Variable Name	Description	Data Type
Input 1	required_modules	The list of needs generated by the Activator.	list<string>
Input 2	completed_modules	The list of modules already finished in this session.	list<string>
Output	next_activity	The ID of the module to launch next.	string

Intern Instructions: How to Write Router Rules

Goal: Define the Clinical Priority and Workflow Logic.

* Sort by Urgency: The manual usually lists "Danger Signs" or "Emergency Triage" first. These must be the top rows. Routine checks (Nutrition, Immunization) go at the bottom. (OR have explicit priority orders?)

* Write the "Emergency Interrupt" Rows (Top Priority):
  * Condition: list contains(required_modules, "mod_referral")
  * Completed Check: Leave Blank. (We don't care if we did it before; if the patient is dying now, we refer now).
  * Output: "mod_referral"
* Write the "Standard Queue" Rows (Middle Priority):
  * Condition: list contains(required_modules, "mod_pneumonia")
  * Completed Check: not(list contains(completed_modules, "mod_pneumonia"))
  * Output: "mod_pneumonia"
  * Note: This ensures we don't repeat the Pneumonia module if we just finished it.
* Write the "End Session" Row (Last Priority):
  * Condition: - (Always True)
  * Completed Check: -
  * Output: "End_Session" (or "Show_Summary")

Example Scenario for Interns

Manual: "Assess for Danger Signs first. Then assess Cough. Then assess Fever. If the child has a Danger Sign, stop assessment and refer."

Activator Table (Unordered):

v_symptoms	v_danger_sign	Output
contains "Cough"	-	mod_cough
contains "Fever"	-	mod_fever
-	True	mod_danger

Router Table (Ordered by Priority):

Priority	Required?	Completed?	Output

1	contains "mod_danger"	-	"mod_referral"
2	contains "mod_cough"	not(contains "mod_cough")	"mod_cough"
3	contains "mod_fever"	not(contains "mod_fever")	"mod_fever"
4	-	-	"End_Session"

* Scenario: Patient has Cough, Fever, AND a Danger Sign.
* Activator: Returns ["mod_cough", "mod_fever", "mod_danger"].
* Router:
  * Row 1 Check: Is mod_danger in the list? Yes.
  * Action: "mod_referral". (Stops immediately. Cough and Fever are skipped).

How the Workflow Executes (The "Movie Script")

Scenario A: The Emergency Case

* Intake: Caregiver says "Cough" (p_req_cough = 1).
* Vitals (Universal): CHW measures MUAC = RED.
  * p_is_emergency becomes 1.
* Router: Checks Rank 0. p_is_emergency is True.
  * Action: INTERRUPT. Jump to Emergency Referral.
  * Result: Cough module is skipped. Life saved.

Scenario B: The Cross-Module Finding

* Intake: Caregiver says "Diarrhea" (p_req_diarrhea = 1). Says "No" to Fever.
* Vitals (Universal): CHW measures MUAC = Green (Normal).
* Router:
  * Rank 0 (Emergency)? No.
  * Rank 1 (Cough)? No.
  * Rank 2 (Diarrhea)? Yes.
  * Action: Run Diarrhea Module.
* Inside Diarrhea Module: CHW measures rectal temp. Result: 38.5°C.
  * System Update: p_req_fever flips to 1.
* Router (Re-run): Diarrhea finishes.
  * Rank 0 (Emergency)? No.
  * Rank 1 (Cough)? No.
  * Rank 2 (Diarrhea)? Already Done.
  * Rank 3 (Fever)? Yes! (Triggered by the temp measurement).
  * Action: Run Fever Module.

Sample module diagnosis table

A sample clinical decision table that a large language model distilled from a CHW training manual

Rule #	Description	Age (months)	Resp. Rate	Chest Indrawing?	Danger Sign?	OUTPUT: Classification
1	Chest indrawing is present	-	-	**TRUE**	-	SeverePneumonia
2	A general danger sign is present	-	-	FALSE	**TRUE**	SeverePneumonia
3	Fast breathing in an infant	>= 2 and < 12	**>= 50**	FALSE	FALSE	Pneumonia
4	Fast breathing in a child	>= 12 and < 60	**>= 40**	FALSE	FALSE	Pneumonia
5	Default case (no severe signs)	-	-	FALSE	FALSE	CoughOrCold

This table was generated entirely by the LLM (formatted in Google Sheets). A medical doctor can easily review. It is in a format called "DMN". Both Python and JavaScript have packages that can interpret DMN directly. We can compile a DMN table into short code snippets that an XLSForm can execute.

Sample treatment table, if decided by each module (mutually exclusive)

	CHW Has amox	**Treatment**	**Dosing table?**
Pneumonia	Yes	Amox	amox_Pneumonia
Pneumonia	No	clinic	N/A
SeverePneumonia	Yes	Amox Clinic	amox_Pneumonia
SeverePneumonia	No	clinic	N/A
coughCold	any	Go home and rest	N/A

Sample of traffic cop #1: Add modules to queue (multiple hits allowed)

	Start?	Has fever?	Has cough?	Has diarrhea?	
At start <<Unclear if in decision table	Yes	any	any	any	Add opening module + integrative module

or just default>>					
Fever	any	Yes	any	any	Add fever module at end of queue, but before integrative
Cough	any	any	Yes	any	Add cough module at end of queue, but before integrative
Diarrhea	any	any	any	Yes	Add diarrhea module at end of queue, but before integrative
Else	No	No	No	No	<<Does manual say?>>

Sample of traffic cop #2: Pick next module (mutually exclusive)

	Queue empty	**urgent referral**	**Next module already done?**	**Result**
Visit done	Yes	Any	Any	End visit
Urgent referral	No	Yes	Any	Go to integrative module (skipping any others)
Next module in queue	No	No	Yes	Remove from queue & restart this table
Next module in queue	No	No	No	Do that module

Sample integrative table

Integrative care module (cross-module integration)

The integrative care module combines outputs from multiple condition modules into a single, coherent care plan. Its purpose is to (1) select a single final disposition/referral level, (2) remove duplicate or conflicting instructions, and (3) resolve medication conflicts and context-specific modifications.

Examples of what a manual might say include:

- Referral priority: Pick "largest" referral, where hospital_urgent > hospital > clinic > home care
- If >1 diagnosis, **deduplicate and group** instructions to reduce cognitive load.
    - Group all home care, follow-up instructions, etc.
    - If > 1 "days till follow-up", pick shortest (or keep both, if the manual says)
        - If the manual does not specify, ask
    - If > 1 home care instruction, list them in order
- Drug interactions
    - If conflicting meds are triggered, apply the **drug-interaction resolver** specified in the manual (e.g., choose A, avoid B, and/or escalate referral).
- Drug duplication, such as:
    - If amoxicillin for skin infection and for pneumonia, keep the larger dose (in days or ml/day?XX)
- Interactions of treatment and facts
    - If treatment = ORS and malnourished, then change the ORS recipe

These are just examples of what to look for; the specifics have to come from the manual.

Additions to prompts

Prompts should be seeking invariants in the model we can feed to the Z3 prover:

- Which tables are UNIQUE policies / mutually exclusive?
- Are there rules such as:
    - No new modules after “Emergency referral”
    - (We do not write this rule, but we have LLMs look for rules such as this.)
- Does every medicine have a dose?
- does every integer and real measure have a unit in its name
- **Solver-generated per-rule witnesses:** for each DMN row, auto-generate at least one satisfying assignment (including missingness/stockout predicates). This guarantees every rule is executable at least once and identifies unreachable rules early.

B6 input underfeeds clinical content

In pipeline_fsm, B6_Decompose_DMN takes only `symbols.json` as input in the snippet,

which risks hallucinated/omitted DMN rules; your red-team table explicitly calls this out.

Even if the prompt text says otherwise, the pipeline wiring is what will run.

Predicate schema is too weak for multi-input predicates
Your schema shows `source_var` singular; real predicates often depend on multiple observations; linters can't validate completeness.

This will eventually break "deterministic compiler" claims because ordering/deps become implicit.

We do not send later prompts to the repaired manual. Add `Resolved_Manual.md` and route B1/B4/B6/D5 to it (repairs + addendum unified).

Prompt D0 to generate invariants for the Z3 prover

- Not yet called by pipeline.YAML.

```
{{header_yaml}}
{{role_redteam}}

# ARTIFACT
artifact_name: invariants.yaml
purpose: A formal, machine-checkable list of correctness properties that MUST hold for the
Activator.dmn + Router.dmn routing logic, plus the predicate layer that feeds them.

# INPUTS (YOU WILL RECEIVE)
1) Resolved_Manual.md  (manual + MOH addendum + any approved repairs)
2) Activator.dmn       (Hit policy: COLLECT)
3) Router.dmn          (Hit policy: FIRST)
4) predicates.json     (definitions of p_* boolean predicates and missingness-safe defaults)
5) symbols.json        (master symbol table: types, namespaces, units)

# OUTPUT (YOU MUST PRODUCE)
Return ONLY valid YAML for invariants.yaml (no markdown, no commentary).
YAML must conform to the schema described below.

# NON-NEGOTIABLE RULES
- DO NOT invent clinical rules. If an invariant is not directly supported by the inputs, mark it
status: "needs_human_decision" and write a question in open_questions[].
```

- Every invariant MUST include provenance: source (file + section/page/line if available) and a short supporting snippet quote (<= 25 words).
- Every invariant MUST be testable against the DMN/predicate layer (i.e., phrased in terms of p_* and c_* inputs and module/queue state variables).
- If a candidate invariant would require reasoning about raw v_* values (e.g., temp > 38), REJECT it and instead require that the predicate already exists (e.g., p_fever_high).
- Prefer invariants that prevent “silent killers”: dead ends, unsafe defaults, contradictory actions, non-termination, and bypass of emergency referral.

```
# REQUIRED YAML SCHEMA (STRICT)
version: "1.0"
generated_from:
  resolved_manual: "Resolved_Manual.md"
  activator_dmn: "Activator.dmn"
  router_dmn: "Router.dmn"
  predicates: "predicates.json"
  symbols: "symbols.json"
invariants:
  - id: INV_###
    title: "short name"
    category: one of [termination, determinism, safety_monotonicity, coverage, consistency,
routing_integrity, stockout_safety, missingness_safety, namespace_contract]
    severity: one of [critical, major, minor]
    status: one of [proposed, needs_human_decision]
    scope:
      applies_to: one of [activator, router, predicates, end_to_end]
      modules: ["mod_*", ...] | ["*"]   # use module IDs that appear in DMN
    statement:
      formal: >
        A single-line formal statement using allowed symbols (see below).
      english: >
        Plain English description of the property and why it matters.
    check_method:
      type: one of [z3, simulation, linter, dmntest]
      inputs_needed: ["Activator.dmn", "Router.dmn", "predicates.json", ...]
      sketch: >
        Pseudocode or precise description of how to check it.
      expected_result: >
        What PASS means. What FAIL means.
    witness_strategy:
      required: true|false
      how_to_generate: >
        If required=true: describe how to generate at least one satisfying (or violating) synthetic
patient / assignment.
```

```
    provenance:
      sources:
        - file: "Resolved_Manual.md" | "Activator.dmn" | "Router.dmn" | "predicates.json"
          locator: "page X / section Y / rule row Z / predicate id"
          snippet: "<=25 words, exact quote"
    notes: "optional"
open_questions:
  - id: Q_###
    question: "What the human/MOH must decide"
    impact_if_unresolved: "why this blocks proving a critical invariant"
    suggested_options: ["option A", "option B", "option C"]
    provenance:
      file: "Resolved_Manual.md"
      locator: "page/section"
      snippet: "<=25 words"
```

ALLOWED SYMBOLS FOR formal STATEMENTS
- Predicates: p_* and context toggles c_*
- Module flags: need_* (if present), required_modules contains("mod_x"), completed_modules contains("mod_x")
- Router output: next_module == "mod_x" or next_state == "END_SESSION"
- Logical operators: AND OR NOT IMPLIES IFF
- Quantifiers (if needed): FORALL, EXISTS over finite sets of modules or boolean assignments (keep minimal)
- Helper notions you may define in-line:
  - emergency_trigger := OR of the emergency predicates used by Router row 1
  - runnable(mod) := required(mod) AND NOT completed(mod)

TASK
1) Extract the module universe:
   - List all module IDs that Router can output (including END_SESSION / referral module/state).
2) Identify the emergency interrupt predicates and referral destinations.
3) Propose a minimal but sufficient set of invariants (aim 12–25):
   - 3–5 termination/determinism
   - 3–5 safety/monotonicity
   - 2–4 coverage (MECE / ELSE rows / unreachable rule detection)
   - 2–4 consistency (no conflicting outputs)
   - 2–4 missingness/stockout safety (fail-safe behavior)
4) For each invariant, provide:
   - A formal statement
   - A concrete check method
   - A witness strategy (at least for critical invariants)
   - Provenance and snippet

QUALITY CHECKS YOU MUST RUN BEFORE OUTPUTTING YAML
QC1: Every invariant references only symbols that exist in symbols.json or predicates.json.
QC2: Every invariant is checkable with the declared check_method type.
QC3: At least one invariant explicitly covers:
- Emergency interrupt short-circuit
- No dead ends (Router always yields a next step)
- No infinite loop / progress (completed set increases or session ends)

QC4: If you mark status=needs_human_decision, you MUST add an entry to open_questions[].

OUTPUT
Return ONLY invariants.yaml as valid YAML.

Testing notes

Testing notes

Product team

- Writes unit testing for each component. Unit tests are small tests for each subroutine, prompt, etc. Unit tests always include at least one test input that should fail gracefully.
    - For example, if you are checking software to enforce a JSON schema, input tests include several legal JSON and at least one failed JSON for each dimension of the definition.
- Writes integration tests

Testing team

- Writes end-to-end tests running simulated patients through
    - DMN
    - Mermaid
    - Z3
    - XLSForm
    - A spectate (e.g., perhaps a gold standard) DMN
    - A gold standard patient record
- Compare outcomes from multiple patient logs coming from the above tests
-

Deploying a clinical software application without exhaustive, automated validation invites catastrophic failure at the point of care. The transition from the conceptual Two-Table architecture to a fully functional Community Health Toolkit deployment requires a multi-layered testing framework, rigorously validating both isolated logical units and end-to-end system integrations.

5.1 Unit Testing the XLSForm Clinical Logic

Unit testing isolates specific rows, calculation fields, and logic gates within the compiled XLSForm to ensure that the mathematical formulas and routing constraints execute precisely as defined by the foundational clinical manual. Due to the immense complexity of deeply nested skip logic, manual regression testing is operationally unscalable and highly susceptible to human error.

The architecture mandates the implementation of automated form validators. Tools such as XFormTest allow developers to execute automated value assertions directly against the compiled XForm XML, ensuring that the syntactic structure correctly represents the clinical intent.

Unit Test Category	Testing Objective	Clinical Execution Example

Constraint Validation	Verifies that the form strictly rejects inputs falling outside defined physiological or operational parameters.	Asserting that an input of 150% for blood oxygen saturation triggers an immediate validation error and halts progression.
Relevancy Branching	Verifies that specific data inputs correctly trigger the display or suppression of subsequent logic groups.	Asserting that entering a patient age of 14 days exclusively renders the neonatal IMCI pathway while suppressing older child modules.
Missingness Handling	Verifies that the logic gracefully processes null inputs without causing systematic calculation failures.	Simulating an unanswered optional question to guarantee that downstream risk score equations do not output a fatal `NaN` exception.
Type Casting Verification	Verifies that boolean states and categorical selections are correctly cast as integers or explicit strings.	Asserting that a `false` evaluation within a string comparison does not trigger the XPath 1.0 boolean truth trap.

5.2 Integration Testing within the CHT Ecosystem

While unit testing validates the isolated form logic, integration testing evaluates how the compiled XLSForm interacts with the broader, complex environment of the Community Health Toolkit. This includes testing interactions with the CouchDB synchronization protocol, state persistence hydration, and user role permission matrices.

The standard, mandated protocol for applications built on this framework requires the use of the `cht-conf-test-harness`. This Node.js-based framework utilizes Mocha and Chai assertion libraries to simulate the exact behavior of the production application instance without requiring the overhead of a full graphical deployment.

Integration Test Scenario	Testing Objective	Simulated Execution Protocol
Form Loading & Rendering	Ensures the application can parse the XML and inject it into the Document	The harness loads the assessment form and asserts that the `pageContent` successfully includes the specific form ID without generating console errors.

	Object Model without fatal syntax errors.	
Longitudinal Hydration	Verifies that historical patient data is successfully retrieved from the database and injected into the active form session.	The harness pre-loads a virtual patient with historical Gravity Project SDOH data, asserting that the `contact-summary` script populates the read-only fields accurately.
Task Generation Mechanics	Verifies the operational bridge between a submitted diagnosis and the automated generation of follow-up tasks.	The harness submits a form indicating suspected malaria, asserting that a follow-up task is correctly generated and placed into the specific health worker's queue.
Role-Based Permissions	Verifies that the application strictly enforces viewing and editing privileges based on the user's assigned hierarchy.	The harness simulates a lower-tier user attempting to access a supervisor-level form, asserting that the system successfully blocks access.

5.3 Automated Headless UI Testing

To guarantee operational stability across highly diverse, low-specification Android hardware, the programmatic backend logic testing must be supplemented with Headless Browser Testing. Utilizing frameworks such as Robot Framework combined with SeleniumLibrary allows the engineering team to simulate physical user interactions—such as scrolling, clicking, and inputting continuous data streams—across the Enketo web interface. This methodology identifies user interface rendering bottlenecks, screen transition delays, and subtle interaction flaws that purely programmatic integration tests might overlook, ensuring the application consistently meets the strict usability requirements dictated by the cognitive load analysis.

A thought on testing

- Run end-to-end once with our team's test harness. So using XLSForm, but not a realistic UI.
- Record the order of questions in the XLSForm.
- Reformat the patient data in that order.
- Run end-to-end once with CHT's test harness.

Inputs for running XLSForm

Put XLSForm CHT into a "box" of some sort. Your test harness captures the XLSForm queries, requests for exams, etc., and sends replies.

- I think this is a headless xxx

Input#1 = XLSForm that has a multilingual phrase bank in it

Set language = variable_name_language

	English	French	Variable_name_language (derived from Emett's spreadsheet)
ex.temp_c	"Measure temperature (°C)"	"Mesure fièvre (°C)"	temp_c
q.cough	"Does the child have a cough?"		cough
dx.pneumonia	"Your diagnosis is pneumonia"		dx.pneumonia
tx.amoxicillin	"Take 1 tsp amox each day for 5 days"		tx.amoxicillin

So CHT XLSForm output queries are "temp_c" or "cough" and diagnoses or treatments

Input#2: Patient data

JSON or CSV or whatever you and medical team agree on.
{cough : no, temp_c : 37.3, female : no}

Capture the XLSForm queries.
Go to patient data, look up matching data

Return the value to CHT

*** WRITE TO THE LOG A BIG ALERT IF AN XLSFORM QUERY HAS NO PATIENT DATA ***
then move on to the next patient.

Output from test harness

Series of questions or exams and values (subset -- often including all -- of this patient's data, probably in a new order) + diagnosis, treatment, follow-up.
{female : no ; cough : no; temp_c : 37.3 ; dx.pneumonia ; tx.amoxicillin}

You may need a rule: "After "dx." Or "tx." Hit "Next" button"

YAML Pipeline

Artifact 3: pipeline.yaml

NOTE: This version does not include Atharva's improvement:

- Makers:
  - LLM1 and LLM2 each stage create the same deliverable.
    - Even better: give slightly different prompts
- Red team
  - LLM3 has a prompt#1 to compare and identify all substantive divergences.
  - LLM3 has prompt#2 to red team the 2 outputs versus the manual and other inputs, identifying where both were wrong
  - LLM3 has prompt#3 to meld its red team report + its divergences report, identifying all problems
- Repair:
  - LLM 4 attempts to repair
- LLM 4 repair and the artifacts from LLM1 and LLM2 go through the red team series until the red team says PASS
  - or 3 fails, in which case humans must help

```
# Pipeline v5.3
# Updated: 2026-02-17
#
# Major changes in this revision:
# - Single truth source: Resolved_Manual.md (manual + MOH addendum +
optional repairs)
# - Traffic-cop logic is now two-table DMN:
#     * Activator.dmn (COLLECT): flags required modules/needs
#     * Router.dmn (FIRST): selects the next module based on required
+ completed + emergency interrupt
# - Removed Priority_Lattice.json as routing IR (optional to
reintroduce later as an internal IR)
#
# Assumptions about runner/tooling:
# - The runner supports transitions with if/else, on_success, and
side_effect (string evaluated by runner).
# - builders.manual.merge_resolved treats missing
manual_draft_repairs.md as empty/no-op.
# - verifiers.dmn.run_reference can run both Activator.dmn and
Router.dmn, or you provide a wrapper tool.

version: "5.1"
project: "Gravity_Gold_Master"

context:
  retry_count: 0
```

```
  max_retries: 3
  enable_translation: true

states:
  #
-----------------------------------------------------------------------
------
  # PHASE A: Authoring & Red Team
  #
-----------------------------------------------------------------------
------

  - id: Render_Prompts
    type: python_code
    tool: builders.prompts.render_all
    inputs: [Prompt_Fragments.csv, Master_Prompts.csv]
    outputs: [rendered_prompts/, rendered_prompts_hashes.json]
    transitions:
      - if: "result.passed"
        target: A1_Make_Addendum
      - else:
        target: HALT_TEMPLATE_ERROR

  - id: A1_Make_Addendum
    type: llm_prompt
    tool: rendered_prompts/A1_make.txt
    inputs: [manual.pdf]
    outputs: [Draft_Addendum.md, Queries.md]
    on_success: A_Human_Gate_Queries

  # MOH (or delegate) resolves ambiguities into a binding addendum.
  - id: A_Human_Gate_Queries
    type: human_gate
    inputs: [Queries.md, Draft_Addendum.md]
    outputs: [final_addendum.md]
    on_success: A2_RedTeam_Audit

  # Red-team the manual + addendum.
  - id: A2_RedTeam_Audit
    type: llm_prompt
    tool: rendered_prompts/A2_make.txt
    inputs: [manual.pdf, final_addendum.md]
    outputs: [manual_redteam_report.md]
    transitions:
      - if: "output.critical_issues == 0"
```

```
        target: Build_Resolved_Manual
      - if: "output.critical_issues > 0 and context.retry_count <
context.max_retries"
        target: A2_Repair_Manual
      - else:
        target: HALT_SAFETY_FAILURE

  # Repair proposes patch text; the patch becomes real only when
merged into Resolved_Manual.md
  - id: A2_Repair_Manual
    type: llm_prompt
    tool: rendered_prompts/A2_repair.txt
    inputs: [manual_redteam_report.md, manual.pdf, final_addendum.md]
    outputs: [manual_draft_repairs.md]
    side_effect: "context.retry_count += 1"
    on_success: Build_Resolved_Manual

  # Build the single source of truth for downstream extraction
(facts, messages, logic).
  - id: Build_Resolved_Manual
    type: python_code
    tool: builders.manual.merge_resolved
    inputs:
      - manual.pdf
      - final_addendum.md
      # optional; merge tool must treat missing file as empty/no-op
      - manual_draft_repairs.md
    outputs: [Resolved_Manual.md]
    transitions:
      - if: "result.passed"
        target: A2_RedTeam_Audit_Resolved
      - else:
        target: HALT_MANUAL_MERGE_ERROR

  # Re-audit the resolved state (closes the “repairs not re-audited”
hole).
  - id: A2_RedTeam_Audit_Resolved
    type: llm_prompt
    tool: rendered_prompts/A2_make_resolved.txt
    inputs: [Resolved_Manual.md]
    outputs: [resolved_manual_redteam_report.md]
    transitions:
      - if: "output.critical_issues == 0"
        target: B1_Extract_Meaning
```

```
      - if: "output.critical_issues > 0 and context.retry_count <
context.max_retries"
        target: A2_Repair_Manual
      - else:
        target: HALT_SAFETY_FAILURE

  #
-----------------------------------------------------------------------
------
  # PHASE B: Meaning Layer (Facts, messages, dictionary, symbols)
  #
-----------------------------------------------------------------------
------

  - id: B1_Extract_Meaning
    type: llm_prompt
    tool: rendered_prompts/B1_make.txt
    inputs: [Resolved_Manual.md]
    outputs: [FactSheet.csv, context_vars.json]
    on_success: Validate_B1

  - id: Validate_B1
    type: python_code
    tool: linters.schemas.validate_b1
    inputs: [FactSheet.csv, context_vars.json]
    transitions:
      - if: "result.passed"
        target: B2_Resolve_Data
      - else:
        target: HALT_SCHEMA_ERROR

  - id: B2_Resolve_Data
    type: llm_prompt
    tool: rendered_prompts/B2_make.txt
    inputs: [FactSheet.csv, WHO_DAK.csv]
    outputs: [Canonical_DataDictionary.csv]
    on_success: Validate_B2

  - id: Validate_B2
    type: python_code
    tool: linters.schemas.validate_dictionary
    inputs: [Canonical_DataDictionary.csv]
    transitions:
      - if: "result.passed"
        target: B4_Enrich_Messages
```

```
      - else:
        target: HALT_SCHEMA_ERROR

  # IMPORTANT: messages come from the same resolved manual as logic.
  - id: B4_Enrich_Messages
    type: llm_prompt
    tool: rendered_prompts/B4_make.txt
    inputs: [FactSheet.csv, Resolved_Manual.md]
    outputs: [message_ids.json]
    on_success: Validate_B4

  - id: Validate_B4
    type: python_code
    tool: linters.schemas.validate_messages
    inputs: [message_ids.json]
    transitions:
      - if: "result.passed"
        target: B_Merge_Symbols
      - else:
        target: HALT_SCHEMA_ERROR

  - id: B_Merge_Symbols
    type: python_code
    tool: compilers.symbol_table.build
    inputs: [Canonical_DataDictionary.csv, context_vars.json,
message_ids.json]
    outputs: [symbols.json]
    transitions:
      - if: "result.passed"
        target: B_Linter_Namespace
      - else:
        target: HALT_SYMBOL_ERROR

  - id: B_Linter_Namespace
    type: python_code
    tool: linters.antigravity.check_names
    inputs: [symbols.json]
    transitions:
      - if: "result.passed"
        target: B5b_Extract_Predicates
      - else:
        target: HALT_NAMESPACE_VIOLATION
```

```
  #
----------------------------------------------------------------------
------
  # PHASE B2: Executable Logic (Predicates, Activator DMN, Router
DMN)
  #
----------------------------------------------------------------------
------

  - id: B5b_Extract_Predicates
    type: llm_prompt
    tool: rendered_prompts/B5b_make.txt
    inputs: [Resolved_Manual.md, FactSheet.csv, context_vars.json,
symbols.json]
    outputs: [predicates.json]
    on_success: Validate_B5b

  - id: Validate_B5b
    type: python_code
    tool: linters.schemas.validate_predicates
    inputs: [predicates.json]
    transitions:
      - if: "result.passed"
        target: B6a_Build_Activator_DMN
      - else:
        target: HALT_SCHEMA_ERROR

  # Activator.dmn: “Clinical Detective” (Hit policy: COLLECT)
produces required modules/needs.
  - id: B6a_Build_Activator_DMN
    type: llm_prompt
    tool: rendered_prompts/B6a_make_activator.txt
    inputs: [Resolved_Manual.md, FactSheet.csv, predicates.json,
symbols.json, context_vars.json]
    outputs: [Activator.dmn]
    on_success: Lint_Activator_DMN

  - id: Lint_Activator_DMN
    type: python_code
    tool: linters.dmn.validate
    inputs: [Activator.dmn]
    transitions:
      - if: "result.passed"
        target: B6b_Build_Router_DMN
      - else:
```

```
        target: HALT_DMN_SYNTAX_ERROR

  # Router.dmn: “Traffic Cop” (Hit policy: FIRST) picks next_module
given required + completed (+ emergency interrupt).
  - id: B6b_Build_Router_DMN
    type: llm_prompt
    tool: rendered_prompts/B6b_make_router.txt
    inputs: [Resolved_Manual.md, FactSheet.csv, predicates.json,
symbols.json, context_vars.json]
    outputs: [Router.dmn]
    on_success: Lint_Router_DMN

  - id: Lint_Router_DMN
    type: python_code
    tool: linters.dmn.validate
    inputs: [Router.dmn]
    transitions:
      - if: "result.passed"
        target: B_Linter_Antigravity_Logic
      - else:
        target: HALT_DMN_SYNTAX_ERROR

  # “Antigravity” subset invariants: DMN uses only p_ and c_ inputs;
else row exists; etc.
  - id: B_Linter_Antigravity_Logic
    type: python_code
    tool: linters.antigravity.check_dmn_subset
    inputs: [Activator.dmn, Router.dmn, symbols.json,
predicates.json]
    transitions:
      - if: "result.passed"
        target: B9_Compile_Calculates
      - else:
        target: HALT_LOGIC_VIOLATION

  # Compile router logic + state management into XLSForm calculates.
  - id: B9_Compile_Calculates
    type: python_code
    tool: compilers.dmn_to_odk.compile_router
    inputs: [Activator.dmn, Router.dmn, predicates.json,
symbols.json]
    outputs: [xlsform_calculates.csv]
    on_success: C1_Build_Survey
```

```
  #
-----------------------------------------------------------------------
------
  # PHASE C: Executable Build
  #
-----------------------------------------------------------------------
------

  - id: C1_Build_Survey
    type: llm_prompt
    tool: rendered_prompts/C1_make.txt
    inputs: [FactSheet.csv, message_ids.json, symbols.json]
    outputs: [xlsform_survey.csv]
    on_success: C2_Merge_Form

  - id: C2_Merge_Form
    type: python_code
    tool: compilers.odk_merger.build_xlsform_and_xml
    inputs: [xlsform_survey.csv, xlsform_calculates.csv]
    outputs: [form.xml]
    on_success: D1b_Gen_Synth_Patients

  #
-----------------------------------------------------------------------
------
  # PHASE D: Verification
  #
-----------------------------------------------------------------------
------

  - id: D1b_Gen_Synth_Patients
    type: python_code
    tool: verifiers.synthetic.generate
    inputs: [predicates.json, symbols.json, CTIO_Standard.json]
    outputs: [synthetic_patients.json]
    on_success: D1c_Run_DMN

  # Run the reference DMN pipeline (Activator then Router).
  - id: D1c_Run_DMN
    type: python_code
    tool: verifiers.dmn.run_reference_activator_router
    inputs: [Activator.dmn, Router.dmn, synthetic_patients.json,
golden_cases.yaml]
    outputs: [dmn_run_results.json]
    on_success: D_Run_Z3
```

```
  # Z3 should prove invariants about routing correctness,
termination, and conflicts.
  # (Recommended future: add invariants.yaml and consume it here
explicitly.)
  - id: D_Run_Z3
    type: python_code
    tool: verifiers.z3_solver.prove_invariants
    inputs: [Activator.dmn, Router.dmn, predicates.json]
    transitions:
      - if: "result.passed"
        target: D4_Run_Robot
      - else:
        target: HALT_MATH_FAILURE

  - id: D4_Run_Robot
    type: python_code
    tool: verifiers.robot.run_headless
    inputs: [form.xml, synthetic_patients.json, golden_cases.yaml]
    outputs: [e2e_results.csv]
    on_success: D_Differential_Check

  - id: D_Differential_Check
    type: python_code
    tool: verifiers.diff.standardizer
    inputs: [dmn_run_results.json, e2e_results.csv,
CTIO_Standard.json]
    transitions:
      - if: "result.passed"
        target: D5_Shadow_Manual
      - else:
        target: HALT_DIVERGENCE

  # Shadow manual for clinician sign-off (should include Mermaid
flowchart generated from Router/queue logic).
  - id: D5_Shadow_Manual
    type: llm_prompt
    tool: rendered_prompts/D5_make.txt
    inputs: [Activator.dmn, Router.dmn, message_ids.json,
Resolved_Manual.md]
    outputs: [Shadow_Manual.md]
    on_success: Final_Human_Gate
```

```
  #
-----------------------------------------------------------------------
------
  # HUMAN GATE: Clinical Logic Sign-off
  #
-----------------------------------------------------------------------
------

  - id: Final_Human_Gate
    type: human_gate
    inputs: [Shadow_Manual.md, Resolved_Manual.md]
    outputs: [clinician_signoff.txt]
    transitions:
      - if: "context.enable_translation"
        target: F1_Gen_PhraseBank
      - else:
        target: G1_Governance

  #
-----------------------------------------------------------------------
------
  # PHASE F: Translation
  #
-----------------------------------------------------------------------
------

  - id: F1_Gen_PhraseBank
    type: llm_prompt
    tool: rendered_prompts/F1_make.txt
    inputs: [message_ids.json]
    outputs: [phrase_bank_en.csv]
    on_success: F2_Translate

  - id: F2_Translate
    type: llm_prompt
    tool: rendered_prompts/F2_make.txt
    inputs: [phrase_bank_en.csv]
    outputs: [phrase_bank_target.csv]
    on_success: F_Linter_Translation

  - id: F_Linter_Translation
    type: python_code
    tool: linters.translation.check_safety
    inputs: [phrase_bank_en.csv, phrase_bank_target.csv,
message_ids.json]
```

```
    transitions:
      - if: "result.passed"
        target: F3_Merge_Translation
      - else:
        target: HALT_TRANSLATION_RISK

  - id: F3_Merge_Translation
    type: python_code
    tool: compilers.odk.merge_languages
    inputs: [form.xml, phrase_bank_target.csv]
    outputs: [form_multilingual.xml]
    on_success: G1_Governance

  #
-----------------------------------------------------------------------
------
  # PHASE G: Governance
  #
-----------------------------------------------------------------------
------

  - id: G1_Governance
    type: python_code
    tool: governance.hasher.generate_manifest
    inputs: [all_artifacts/]
    outputs: [Manifest.json, Governance_Log.md]
    on_success: READY_FOR_DEPLOY
```

If you want one more “hardening” pass (optional but high value), the next improvement would be to add an explicit `invariants.yaml` artifact (generated + human-readable) and require Z3 to consume it—so proofs can’t silently drift.

Old YAML

Old YAML

Artifact 3: pipeline.yaml

NOTE: This version does not include Atharva's improvement:

- Makers:
  - LLM1 and LLM2 each stage create the same deliverable.
    - Even better: give slightly different prompts
- Red team
  - LLM3 has a prompt#1 to compare and identify all substantive divergences.
  - LLM3 has prompt#2 to red team the 2 outputs versus the manual and other inputs, identifying where both were wrong
  - LLM3 has prompt#3 to meld its red team report + its divergences report, identifying all problems
- Repair:
  - LLM 4 attempts to repair
- LLM 4 repair and the artifacts from LLM1 and LLM2 go through the red team series until the red team says PASS
  - or 3 fails, in which case humans must help

YAML

None

```
version: "5.1"
project: "Gravity_Gold_Master"
context:
  retry_count: 0
  max_retries: 3
  enable_translation: true

states:
  # --- PHASE A: Authoring & Red Team ---
  - id: Render_Prompts
    type: python_code
    tool: builders.prompts.render_all
    inputs: [Prompt_Fragments.csv, Master_Prompts.csv]
    outputs: [rendered_prompts/,
rendered_prompts_hashes.json]
    transitions:
```

```
      - if: "result.passed"
        target: A1_Make_Addendum
      - else:
        target: HALT_TEMPLATE_ERROR

  - id: A1_Make_Addendum
    type: llm_prompt
    tool: rendered_prompts/A1_make.txt
    inputs: [manual.pdf]
    outputs: [Draft_Addendum.md, Queries.md]
    on_success: A_Human_Gate_Queries

  - id: A_Human_Gate_Queries
    type: human_gate
    inputs: [Queries.md, Draft_Addendum.md]
    outputs: [final_addendum.md]
    on_success: A2_RedTeam_Audit

  - id: A2_RedTeam_Audit
    type: llm_prompt
    tool: rendered_prompts/A2_make.txt
    inputs: [manual.pdf, final_addendum.md]
    outputs: [manual_redteam_report.md]
    transitions:
      - if: "output.critical_issues == 0"
        target: B1_Extract_Context
      - if: "output.critical_issues > 0 and
context.retry_count < context.max_retries"
        target: A2_Repair_Manual
      - else:
        target: HALT_SAFETY_FAILURE

  - id: A2_Repair_Manual
    type: llm_prompt
    tool: rendered_prompts/A2_repair.txt
```

```
    inputs: [manual_redteam_report.md, manual.pdf]
    outputs: [manual_draft_repairs.md]
    side_effect: "context.retry_count += 1"
    on_success: A2_RedTeam_Audit

  # --- PHASE B: Meaning & Logic (Reordered) ---
  - id: B1_Extract_Context
    type: llm_prompt
    tool: rendered_prompts/B1_make.txt
    inputs: [manual.pdf, final_addendum.md,
manual_draft_repairs.md]
    outputs: [FactSheet.csv, context_vars.json]
    on_success: Validate_B1

  - id: Validate_B1
    type: python_code
    tool: linters.schemas.validate_b1
    inputs: [FactSheet.csv, context_vars.json]
    transitions:
      - if: "result.passed"
        target: B2_Resolve_Data
      - else:
        target: HALT_SCHEMA_ERROR

  - id: B2_Resolve_Data
    type: llm_prompt
    tool: rendered_prompts/B2_make.txt
    inputs: [FactSheet.csv, WHO_DAK.csv]
    outputs: [Canonical_DataDictionary.csv]
    on_success: Validate_B2

  - id: Validate_B2
    type: python_code
    tool: linters.schemas.validate_dictionary
    inputs: [Canonical_DataDictionary.csv]
```

```
    transitions:
      - if: "result.passed"
        target: B4_Enrich_Messages
      - else:
        target: HALT_SCHEMA_ERROR

  - id: B4_Enrich_Messages
    type: llm_prompt
    tool: rendered_prompts/B4_make.txt
    inputs: [FactSheet.csv, Manual.pdf]
    outputs: [message_ids.json]
    on_success: Validate_B4

  - id: Validate_B4
    type: python_code
    tool: linters.schemas.validate_messages
    inputs: [message_ids.json]
    transitions:
      - if: "result.passed"
        target: B_Merge_Symbols # <-- SYMBOLS BUILT
HERE
      - else:
        target: HALT_SCHEMA_ERROR

  - id: B_Merge_Symbols
    type: python_code
    tool: compilers.symbol_table.build
    inputs: [Canonical_DataDictionary.csv,
context_vars.json, message_ids.json]
    outputs: [symbols.json]
    transitions:
      - if: "result.passed"
        target: B_Linter_Namespace
      - else:
        target: HALT_SYMBOL_ERROR
```

```yaml
  - id: B_Linter_Namespace
    type: python_code
    tool: linters.antigravity.check_names
    inputs: [symbols.json]
    transitions:
      - if: "result.passed"
        target: B5b_Extract_Predicates # <-- PREDICATES
BUILT AFTER SYMBOLS
      - else:
        target: HALT_NAMESPACE_VIOLATION

  - id: B5b_Extract_Predicates
    type: llm_prompt
    tool: rendered_prompts/B5b_make.txt
    inputs: [FactSheet.csv, symbols.json]
    outputs: [predicates.json]
    on_success: Validate_B5b

  - id: Validate_B5b
    type: python_code
    tool: linters.schemas.validate_predicates
    inputs: [predicates.json]
    transitions:
      - if: "result.passed"
        target: B6_Decompose_DMN
      - else:
        target: HALT_SCHEMA_ERROR

  - id: B6_Decompose_DMN
    type: llm_prompt
    tool: rendered_prompts/B6_make.txt
    inputs: [symbols.json]
```

```
  xx FactSheet.csv + predicates.json + message_ids.json +
     Resolved_Manual.md excerpts and require per-row
     traceability.

    outputs: [DMN_Modules.xml]
    on_success: B_Linter_Antigravity

  - id: B_Linter_Antigravity
    type: python_code
    tool: linters.antigravity.check_invariants
    inputs: [DMN_Modules.xml, symbols.json]
    transitions:
      - if: "result.passed"
        target: B7_Integrative_DMN
      - else:
        target: HALT_LOGIC_VIOLATION

  - id: B7_Integrative_DMN
    type: llm_prompt
    tool: rendered_prompts/B7_make.txt
    inputs: [DMN_Modules.xml, FactSheet.csv,
symbols.json]
    outputs: [DMN_Integrative.xml,
Priority_Lattice.json]
    on_success: Validate_B7
  - id: Validate_B7
    type: python_code
    tool: linters.schemas.validate_lattice
    inputs: [Priority_Lattice.json,
std_priority_lattice_schema.json]
    transitions:
      - if: "result.passed"
        target: B9_Compile_Calculates
      - else:
```

```
        target: HALT_SCHEMA_ERROR

  - id: B9_Compile_Calculates
    type: python_code
    tool: compilers.dmn_to_odk.compile
    inputs: [DMN_Integrative.xml,
Priority_Lattice.json, predicates.json]
    outputs: [xlsform_calculates.csv]
    on_success: C1_Build_Survey

  # --- PHASE C: Executable Build ---
  - id: C1_Build_Survey
    type: llm_prompt
    tool: rendered_prompts/C1_make.txt
    inputs: [FactSheet.csv, message_ids.json,
symbols.json]
    outputs: [xlsform_survey.csv]
    on_success: C2_Merge_Form

  - id: C2_Merge_Form
    type: python_code
    tool: compilers.odk_merger.build_xlsform_and_xml
    inputs: [xlsform_survey.csv,
xlsform_calculates.csv]
    outputs: [form.xml]
    on_success: D1b_Gen_Synth_Patients

  # --- PHASE D: Verification ---
  - id: D1b_Gen_Synth_Patients
    type: python_code
    tool: verifiers.synthetic.generate
    inputs: [predicates.json, symbols.json,
CTIO_Standard.json]
    outputs: [synthetic_patients.json]
    on_success: D1c_Run_DMN
```

```yaml
  - id: D1c_Run_DMN
    type: python_code
    tool: verifiers.dmn.run_reference
    inputs: [DMN_Modules.xml, DMN_Integrative.xml,
synthetic_patients.json, golden_cases.yaml]
    outputs: [dmn_run_results.json]
    on_success: D_Run_Z3

  - id: D_Run_Z3
    type: python_code
    tool: verifiers.z3_solver.prove_invariants
    inputs: [DMN_Modules.xml, predicates.json]
    transitions:
      - if: "result.passed"
        target: D4_Run_Robot
      - else:
        target: HALT_MATH_FAILURE

  - id: D4_Run_Robot
    type: python_code
    tool: verifiers.robot.run_headless
    inputs: [form.xml, synthetic_patients.json,
golden_cases.yaml]
    outputs: [e2e_results.csv]
    on_success: D_Differential_Check

  - id: D_Differential_Check
    type: python_code
    tool: verifiers.diff.standardizer
    inputs: [dmn_run_results.json, e2e_results.csv,
CTIO_Standard.json]
    transitions:
      - if: "result.passed"
        target: D5_Shadow_Manual
```

```
      - else:
        target: HALT_DIVERGENCE

  - id: D5_Shadow_Manual
    type: llm_prompt
    tool: rendered_prompts/D5_make.txt
    inputs: [DMN_Integrative.xml,
Priority_Lattice.json, message_ids.json]
    outputs: [Shadow_Manual.md]
    on_success: Final_Human_Gate

  - id: Final_Human_Gate
    type: human_gate
    inputs: [Shadow_Manual.md, manual.pdf]
    outputs: [clinician_signoff.txt]
    transitions:
      - if: "context.enable_translation"
        target: F1_Gen_PhraseBank
      - else:
        target: G1_Governance

  # --- PHASE F: Translation ---
  - id: F1_Gen_PhraseBank
    type: llm_prompt
    tool: rendered_prompts/F1_make.txt
    inputs: [message_ids.json]
    outputs: [phrase_bank_en.csv]
    on_success: F2_Translate

  - id: F2_Translate
    type: llm_prompt
    tool: rendered_prompts/F2_make.txt
    inputs: [phrase_bank_en.csv]
    outputs: [phrase_bank_target.csv]
    on_success: F_Linter_Translation
```

```
  - id: F_Linter_Translation
    type: python_code
    tool: linters.translation.check_safety
    inputs: [phrase_bank_en.csv,
phrase_bank_target.csv, message_ids.json]
    transitions:
      - if: "result.passed"
        target: F3_Merge_Translation
      - else:
        target: HALT_TRANSLATION_RISK

  - id: F3_Merge_Translation
    type: python_code
    tool: compilers.odk.merge_languages
    inputs: [form.xml, phrase_bank_target.csv]
    outputs: [form_multilingual.xml]
    on_success: G1_Governance

  # --- PHASE G: Governance ---
  - id: G1_Governance
    type: python_code
    tool: governance.hasher.generate_manifest
    inputs: [all_artifacts/]
    outputs: [Manifest.json, Governance_Log.md]
    on_success: READY_FOR_DEPLOY
```

Critiques

Control-flow logic defects (these matter for correctness)

3. **Repairs loop doesn't feed repairs into re-audit** (silent killer)
   A2_RedTeam_Audit inputs are `[manual.pdf, final_addendum.md]` but after A2_Repair_Manual produces `manual_draft_repairs.md`, the pipeline loops back to A2_RedTeam_Audit without including repairs in the audit inputs.
   CHW Navigator v1
   CHW Navigator v1

   Result: you can iterate forever without the red team ever assessing the patched state; or you proceed with "repairs" that never affect anything upstream.
   Fix: introduce an explicit `Apply_Repairs` state that creates `Resolved_Manual.md` (manual + addendum + repairs), then re-audit *that*.
4. **"Truth source" inconsistency between facts and messages** (silent killer)
   B1 consumes `manual.pdf + final_addendum.md + manual_draft_repairs.md`, but B4 (messages) consumes only `Manual.pdf`.

   So the compiled logic may reflect addendum/repairs while the text shown to CHWs reflects the original manual (or vice versa). This can pass all equivalence tests and still be unsafe.
   Fix: make `Resolved_Manual.md` the **only** source for B1/B4/B6/B7/D5. (Your embedded red team says exactly this.
   CHW Navigator v1
)
5. **Optional artifact handling is undefined**
   If `output.critical_issues == 0`, the pipeline never produces `manual_draft_repairs.md`, but B1 lists it as an input.

   Depending on your runner, this becomes either a crash or "use stale file from last run."
   Fix: define `manual_draft_repairs.md` as always existing (possibly empty), or branch B1 inputs based on whether repairs exist.

C. "Route"/wiring mismatch issues (developer note)

Since you flagged "route" specifically: there's a naming/wiring issue that will confuse people implementing the runner.

6. **Multiple step styles: `on_success` vs `transitions`**
   Some states use `on_success: NextState`, others use `transitions` with `if:` expressions.
   CHW Navigator v1

That's fine if your runner supports both, but if "route" is a concern: pick one convention so contributors don't accidentally omit failure routing.

7. **Ambiguous meaning of `result` vs `output`**
   Render_Prompts routes on `result.passed`. A2_RedTeam_Audit routes on `output.critical_issues`.
   [CHW Navigator v1]

   That's ok if your executor standardizes return shapes, but you should document the contract: every tool must return either `result` or `output` with a known schema. Otherwise routing becomes fragile.

D. Build completeness and safety (not YAML syntax, but pipeline correctness)

8. **Z3 proof scope is unclear and can be misleading**
   `D_Run_Z3` proves invariants using `[DMN_Modules.xml, predicates.json]`.

   But your standards acknowledge XPath/ODK semantics differ from Z3 unless modeled; treat Z3 as bug-finder unless semantics aligned.

   Fix: either (a) explicitly label Z3 as "bug-finding" not "sound proof," or (b) formalize your subset semantics and encode them.

Sample Schemas

Artifact 5: Schema Library

A. The Symbol Table Schema (`std_symbols_schema`)

Note that our variable naming convention is more complete. XX

JSON

None

```
{
  "title": "Master Symbol Table",
  "type": "array",
  "items": {
    "type": "object",
    "required": ["canonical_id", "odk_name", "scope",
"type", "provenance"],
    "properties": {
      "canonical_id": { "type": "string",
"description": "e.g., patient.vitals.temp" },
      "odk_name": {
        "type": "string",
        "pattern": "^(p_|v_|c_|m_)[a-z0-9_]+$",
        "description": "Must use Hungarian Prefix"
      },
      "scope": { "enum": ["patient", "context",
"message", "predicate"] },
      "type": { "enum": ["string", "integer",
"decimal", "select_one", "boolean"] },
      "units": { "type": "string", "enum": ["degC",
"kg", "months", "days", "count", "none"] },
      "provenance": {
        "type": "object",
        "required": ["source_artifact", "page_ref"],
        "properties": {
          "source_artifact": { "type": "string" },
          "page_ref": { "type": "string" },
          "snippet": { "type": "string" }
```

```
          }
        }
      }
    }
  }
```

B. The Phrase Bank Schema (`std_phrasebank_schema`)

JSON

None

```
{
  "title": "Master Phrase Bank",
  "type": "array",
  "items": {
    "type": "object",
    "required": ["message_id", "language_code", "text",
"status"],
    "properties": {
      "message_id": { "type": "string", "pattern":
"^m_[a-z0-9_]+$" },
      "language_code": { "type": "string", "enum":
["en", "fr", "sw", "es"] },
      "text": { "type": "string" },
      "status": { "enum": ["draft", "verified",
"flagged"] },
      "translator_id": { "type": "string" },
      "flagged_reason": { "type": "string" }
    }
  }
}
```

C. The Context Schema (`std_context_schema`)

JSON

None

```
{
  "title": "Context Variables",
  "type": "object",
  "patternProperties": {
    "^c_[a-z0-9_]+$": {
      "type": "object",
      "required": ["value", "dtype", "source"],
      "properties": {
        "value": { "type": ["string", "integer",
"boolean"] },
        "dtype": { "enum": ["const", "setting",
"toggle"] },
        "source": { "type": "string" }
      }
    }
  }
}
```

D. The Predicate Schema (`std_predicates_schema`)

These are boolean variables used in decisions.

JSON

None

```
{
  "title": "Predicate Definitions",
  "type": "array",
  "items": {
    "type": "object",
    "required": ["id", "source_var", "xpath_present",
"xpath_logic", "fail_safe_value"],
```

```
    "properties": {
      "id": { "type": "string", "pattern":
"^p_[a-z0-9_]+$" },
      "source_var": { "type": "string", "pattern":
"^v_[a-z0-9_]+$" },
      "xpath_present": { "type": "string" },
      "xpath_logic": { "type": "string" },
      "fail_safe_value": { "type": "integer", "enum":
[0, 1] }
    }
  }
}
```

E. The FactSheet Schema (`std_factsheet_schema`)

JSON

None

```
{
  "title": "FactSheet",
  "type": "array",
  "items": {
    "type": "object",
    "required": ["fact_id", "category", "content",
"priority_weight", "source"],
    "properties": {
      "fact_id": { "type": "string" },
      "category": { "enum": ["symptom", "action",
"drug", "workflow"] },
      "content": { "type": "string" },
      "priority_weight": { "type": "integer",
"minimum": 1 },
      "source": {
        "type": "object",
```

```
        "required": ["manual_id", "page", "snippet"],
        "properties": {
          "manual_id": { "type": "string" },
          "page": { "type": "integer" },
          "snippet": { "type": "string" }
        }
      }
    }
  }
}
```

F. The Synthetic Patient Schema (`std_synthetic_patient_schema`)

JSON

None

```
{
  "title": "Synthetic Patient Definition",
  "type": "object",
  "required": ["case_id", "scenario_type", "data"],
  "properties": {
    "case_id": { "type": "string" },
    "scenario_type": { "enum": ["boundary",
"missingness", "stockout", "happy_path"] },
    "data": {
      "type": "object",
      "patternProperties": {
        "^(v_|c_)[a-z0-9_]+$": { "type": ["string",
"number", "boolean", "null"] }
      },
      "additionalProperties": false
    }
  }
```

```
}
```

Red Team

These are a bunch of LLM critiques of the plan.

ChatGPT red team

No-go as written. The architecture is directionally strong (single-source artifacts, deterministic compile, differential DMN↔ODK testing), but v5.1 contains several “silent killers” where the pipeline can pass while the deployed form behaves unsafely or unreviewably. Below is a rigorous red-team focused on failure modes that would not necessarily crash the build.

1. Top “silent killers” (can pass CI, still harm patients)
ID	Failure mode	Why it’s dangerous	Where it appears	Why it can slip through	Hard fix (not just “be careful”)
SK-1	Missingness fail-safe triggers before questions are asked (“blank because not asked yet” == “missing measurement”)	Can short-circuit to END_REFERRAL immediately, or route incorrectly, before collecting danger signs/vitals; yields pervasive over-referral or wrong triage	std_missingness_model + queue emergency exit + ODK calculates	Unit tests may be written assuming fields already asked; E2E robot may fill in fast and never hit pre-ask states	Introduce explicit “asked” state per observation (e.g., p_asked_temp=1 once question displayed/answered), and gate predicates/router on asked-status. Only apply fail-safe once asked or explicitly “unable to measure.”
SK-2	Emergency Exit “no further questions asked” not enforced at UI level	Even if DMN says END_REFERRAL, the form may still show/allow other modules due to relevance/ordering errors	std_safe_endpoints + B9/C1 design	Differential check can match DMN/ODK outputs yet still show extra questions in UI	Add a global kill-switch relevance condition: if p_emergency==1 then all non-referral groups irrelevant. Add a robot test asserting no non-emergency questions are displayed after trigger.
SK-3	completed_bits uses contains() on raw slugs (substring collisions)	Can mark wrong module complete (skip critical module) or loop modules	WriteOnce_Code A + B9 prompt	Tests might not include colliding slugs; collisions are “data-dependent” on naming	Use delimiter-guarded set: store `|respiratory|diarrhea|` and test with `contains(${completed_bits}, concat('|', ${module}, '|'))`. Make append idempotent.
SK-4	Back-navigation/app resume repeatedly appends module slugs	completed_bits grows; may change routing or performance; can create unexpected “complete” behavior after partial runs	WriteOnce_Code A	Robot runs are linear; real users go back/forward	Append only if not already present; also cap/normalize the set string.
SK-5	B6 DMN decomposition lacks clinical inputs (symbols.json only)	DMN rules can be hallucinated/omitted; downstream equivalence tests won’t catch if both reference/runtime share same wrong intent	Master_Prompts B6_make + pipeline B6_Decompose_DMN inputs	Linter checks syntax/invariants, not clinical completeness	Feed B6 at minimum: FactSheet.csv + predicates.json + message_ids.json + resolved manual excerpts. Require every DMN row to cite FactSheet fact_id(s). Block build if uncited.
SK-6	Repairs loop doesn’t feed repairs into re-audit	You will never converge on a safer manual; or you will “think” you did but red-team never re-evaluates the applied patch	pipeline_fsm.yaml: A2_Repair_Manual → A2_RedTeam_Audit (but audit inputs exclude repairs)	Pipeline may HALT or, worse, proceed with unincorporated fixes	Create an explicit “Apply_Repairs” state producing a single Resolved_Manual.md (manual + addendum + repairs). Re-audit *that* artifact.

SK-7	Facts/messages extracted from different “truth” sources	Logic may reflect addendum/repairs while messages reflect original manual (or vice versa)	B1 uses final_addendum + repairs; B4 uses Manual.pdf only	Everything compiles; clinicians see inconsistent behavior	Make Resolved_Manual.md the only clinical source for B1/B4/B6/B7/D5. Enforce in schema: every message has provenance into Resolved_Manual.md.
SK-8	Predicate schema can’t represent multi-input predicates (only one source_var)	Encourages under-specification; dependency cycles/toposort errors become invisible; real clinical logic often uses multiple observations	std_predicates_schema	xpath_logic may reference extra vars but schema doesn’t declare them; linters can’t prove completeness	Change schema: `source_vars: [v_...]`, `depends_on: [p_...]`, `context_deps: [c_...]`. Build a dependency DAG and topologically order calculates from that, not “best effort.”
SK-9	CTIO contract mismatch (HALT/next_module not in CTIO; synthetic schema differs)	Verification can give false passes because you’re comparing different fields or dropping them	std_ctio_schema vs comparator vs synthetic schema	Standardizer may silently ignore missing keys	Update CTIO to include next_module + halt_reason; align golden_cases.yaml schema with CTIO; validate strict equality including routing.
SK-10	Type system mismatch: DMN booleans vs ODK 1/0	A predicate that is “1” may not equal FEEL true; rows may never hit; or compiler may coerce unexpectedly	std_dmn_subset + predicate 1/0 convention	Tests may not cover coercion edges; FEEL/XPath differences	Normalize: DMN inputs are boolean only; emit p_*bool* = *(p* = 1) and feed those to DMN, or switch predicates to true/false consistently end-to-end (but then fully solve the ODK boolean trap). Don’t mix.
SK-11	Z3 “proof” can be unsound relative to XPath/ODK semantics	“Proved invariant” may not hold on device (nulls, string→number coercions, select_one behavior)	D_Run_Z3 stage	Passing proof becomes a false safety signal	Treat Z3 as bug-finder only unless you formally model XPath semantics used. Require independent runtime property tests (metamorphic tests) on the actual compiled form.
SK-12	Translation safety checks only placeholders, not clinical meaning	A mistranslation of “refer urgently” vs “routine referral” kills	Phase F	Linter passes if tokens intact	Add bilingual clinician gate for all “critical severity” messages; require back-translation review for flagged strings; keep per-message risk class and gate by it.

2. Build-stoppers and “looks small, becomes catastrophic”
   These are not clinical per se, but they will cause brittle operations and “unknown unknowns” in emergencies.
Issue	What’s wrong	Impact	Fix
YAML contains `***`	Invalid YAML	Pipeline may not run or may be mis-parsed	Remove; add YAML schema validation step before any run.
Manual.pdf vs manual.pdf	Case mismatch	Works on one OS, fails on another	Canonicalize filenames; add a preflight that asserts all referenced artifacts exist exactly.
Missing prompt fragments referenced (e.g., std_data_quality, std_supplies_ops, std_schema_validation, footer_safety_phi, role_qa_engineer, role_translator, role_governance, F2_make)	Render step may pass if not strict, or later steps run with missing standards	Hidden regression in safety constraints	Make render step fail hard on any unresolved template token. Produce a report of all fragment IDs used.
Optional artifacts not handled (manual_draft_repairs.md when no issues)	Downstream

steps may fail or silently use stale files | Non-reproducibility | Explicitly model “empty repairs” as a real artifact (e.g., manual_repairs_applied.md = empty but valid). |

3. The single biggest conceptual risk: self-referential correctness
   You’re doing DMN reference, compiling to ODK, and then checking equivalence between them. That detects compiler bugs, not “wrong guideline encoded.” If the guideline extraction is wrong upstream, both DMN and ODK will agree and you’ll get a clean bill of health.

Your only external oracle is golden_cases.yaml + clinician sign-off on Shadow_Manual.md. That’s good, but insufficient unless you harden it:
Minimum hardening for the oracle

1. golden_cases.yaml must be authored/owned by clinicians (not LLM-generated), versioned, and expanded until it covers every DMN rule row at least once.

2. Require traceability: every DMN row must cite FactSheet fact_id(s), and every fact_id must cite exact provenance (page+snippet). Block build if traceability is missing.

3. Add “negative oracle” tests: cases that must NOT trigger certain actions (e.g., no antibiotic if condition absent), to catch overbroad rules.

4. Critical schema/contract inconsistencies to fix before trusting any verifier
   A) Disposition enum mismatch
   You define endpoints as REFER, TREAT, OBSERVE, HALT, but CTIO only allows REFER|TREAT|OBSERVE. Either HALT is real (then CTIO must allow it) or HALT is an internal engineering state that must never reach CHW outputs (then enforce that invariant and remove from endpoints list).

B) Synthetic vs CTIO mismatch
Synthetic patient schema holds only v_/c_ in `data`, while CTIO expects nested observations/predicates/context. Decide one canonical representation and enforce it everywhere. If you keep both, write a single, tested converter and validate round-trips.

D) Context values types disagree
std_context_schema allows string/integer/boolean, but CTIO context is 0/1. If context includes non-binary values (e.g., zone codes), CTIO must support them and DMN must model them safely (or map to predicates p_is_zone_X).

5. Concrete changes that would materially improve safety (priority order)
   P0 (must do before any field trial)

1. Create Resolved_Manual.md (single source) and route all “meaning extraction” (B1/B4/B6/B7/D5) to it. Enforce provenance to this artifact everywhere.

2. Fix the repairs loop: A2_Repair_Manual must feed into Resolved_Manual.md, then re-run A2 audit on Resolved_Manual.md. Add human gate “Apply repairs” if needed.

3. Redesign missingness: distinguish NOT_ASKED vs ASKED_BUT_MISSING vs UNABLE_TO_MEASURE. Apply fail-safe only in the latter two.

4. Replace completed_bits logic with delimiter-safe, idempotent set. Add a linter that rejects module IDs that are substrings of other IDs (belt-and-suspenders).

5. Make all LLM outputs schema-validated artifacts, not free text. Any parse failure halts.

6. Align CTIO, golden cases, and differential comparator fields (include next_module; decide HALT).

P1 (next)
7. Upgrade predicate schema to declare dependencies; build a dependency graph; refuse cycles; topologically order calculates from the graph.
8. Add DMN XML validation + “MECE coverage” checker that proves an ELSE row exists and the table covers all boolean combinations for its declared inputs (or flags the uncovered combos).
9. Add UI-level tests: robot asserts that after emergency trigger, no non-emergency questions are visible; and that every path reaches a terminal disposition.

P2 (ongoing operations)
10. Translation risk classing + bilingual clinician gate for critical messages.
11. Stockout handling as a runtime input (or daily facility config), not a static compile-time constant. Default behavior when unknown must be explicit and conservative, and visible in the final message (“stock unknown; refer”).
12. Post-deployment monitoring: log predicate snapshots + outputs (privacy-safe) to detect drift and systematic over/under referral.

6. Two critiques of the overall approach (even if you implement the fixes)
   Critique 1: LLM-mediated extraction remains the dominant risk
   Even with strong schemas, an LLM can omit a single condition or invert an age band. Differential DMN↔ODK tests won’t catch shared upstream errors. Your safety depends on the strength of the oracle (golden cases + clinician review), which is typically the first thing to under-resource.

Critique 2: “Fail-safe to higher risk” can create system-level harm
Always escalating on missingness is individually conservative but can swamp referral centers, reduce trust, and incentivize CHWs to game the form (entering fake values to avoid referral). That can paradoxically increase mortality. You need an explicit operational safety model (capacity-aware pathways, “measure now” prompts, equipment-failure workflows) rather than a blanket rule.

7. Two creative, high-leverage ideas (worth doing given the stakes)
   Idea 1: N-version guideline compilation (independent pipelines)
   Run two independent “meaning → logic” pipelines (e.g., different models/prompts or one LLM-based and one rules/annotator-based), then diff DMN tables at the rule level. Any disagreement becomes a mandatory clinician adjudication. This attacks the shared-upstream-error problem directly.

Idea 2: SMT-based row-coverage test generation from DMN (not just boundary triplets)
For each DMN row, use a solver to generate at least one satisfying assignment of its boolean inputs (including missingness/stockout predicates). This guarantees every rule is executed at least once, and it will also find unreachable rows (a classic “looks fine, never fires” killer).

Bottom line
The “Gold Master” concept can work, but v5.1 as specified is not deployable in a life-critical

setting without P0 fixes. The most dangerous issues are (1) missingness semantics before questions are asked, (2) weak state persistence via contains(), (3) self-referential verification that can bless shared upstream errors, and (4) repair/addendum inconsistencies that fracture the “single source of truth.”

Possible additions

1. **Fail-safe escalation can create system harm:** always escalating on missingness is individually conservative but can overload referrals, reduce trust, and incentivize gaming. You need an explicit operational pathway for “not asked yet” vs “unable to measure” vs “measurement missing.”

1. The "State Persistence" Gap (The Biggest Risk)

Source Reality: Guidelines often say: "If the child had fever yesterday..." or "If the child is currently on Amoxicillin..."

Current Plan: Your synthetic_patient.json has a snapshot of "Inputs," but your ODK Compiler focuses on the current session.

The Failure Mode: The logic works for a single visit, but the app has amnesia. It fails to retrieve the "History" variable from the CHT/OpenSRP database, leading to incorrect treatment (e.g., giving a second dose too soon).

The Fix:

* Add Artifact: persistence_schema.json.

* Definition: Explicitly maps which variables must be saved to the device database and which must be loaded at the start of the next session.

* Compiler Task: The compiler must generate the save_to_db and load_from_db directives specific to the target platform (CHT vs. OpenSRP).

2. The "Cognitive Overload" Gap (UX Risk)

Source Reality: A "Complete" DMN might cover 500 edge cases.

Current Plan: The Compiler deterministically converts all logic into ODK.

The Failure Mode: The resulting form is 80 screens long. The CHW, frustrated by the length, starts entering "No" for everything just to finish the visit. The system is "Safe" mathematically but "Unsafe" behaviorally.

The Fix:

* Add Artifact: UX_Constraints.json (Phase 0).

* Definition: Sets budgets for "Clicks per Workflow," "Screens per Module," and "Maximum Nesting Depth."

Better: Make cognitive load = 1 for Boolean, 0.5 for information only screen, 2 for pick from short list or enter a #, 5 for an exam. Then score = total cognitive load of a path. Create constraints (derived from User research)

- One module path always < 25 points for example
- 2 module path < 45
- worst case < ??? (This will be very rare, but we also want to ensure the kid is referred!)

Then either using the prover or just logs from 1000 synthetic patients, ensure the constraints are always met (or permit some % failure?)

* Verifier Task: The "Graph Walker" (Runner C) or whatever simulator we use must fail the build if the "Happy Path" (Routine Visit) exceeds 25 cognitive load points.

3. The "Media & Literacy" Gap

Source Reality: CHWs often have low literacy. They rely on images/audio, not just text. MOH guidelines define what to do, but not how to show it.

Current Plan: You have a Message_Registry.json (Text), but no Media_Map.

The Failure Mode: The app displays perfect clinical text ("Assess for visible severe wasting"), but the CHW doesn't understand the term.

The Fix:

* Add Artifact: Media_Map.csv.

* Definition: Maps message_id to image_filename (e.g., img_wasting_illustration.png) and audio_filename.

* Compiler Task: Verify that every referenced image actually exists in the /media folder before building the APK.

Revised Phase 0 Artifact List (The "Bulletproof" Version)

To guarantee success, add the items marked [NEW] to your schema definition.

1. Core Logic (The Brain)

* symbols.json (The Linker)

* FactSheet.csv (The Source)

* Priority_Lattice.json (The Router)

* Clinical_Trigger_Map.csv (The Activator)

* [NEW] persistence_schema.json (The Memory): Defines write_back (save to DB) and pre_load (read from DB) variables.

2. Content & UX (The Face)

* Message_Registry.json (Text & Translation)

* [NEW] Media_Map.csv (Visuals): Columns: message_id, image_path, audio_path, video_path.

* [NEW] UX_Constraints.json (The Budget): max_screens_per_flow, required_image_frequency.

3. Verification (The Test Rig)

* synthetic_patient.json (Current State)

* [UPDATE] synthetic_history.json (Longitudinal State): Adds the "Time Dimension" (e.g., "Visit 1: Malaria +", "Visit 2: ?").

* env_context.json (The World: Stockouts, Time)

* expected_case.json (The Truth)

4. Platform Adapters (The "Last Mile")

* [NEW] Platform_Config.yaml:

  * Target: CHT | ODK | OpenSRP.

  * Version: 4.2.

  * Specific Compiler Flags: (e.g., "Use properties for CHT, use instances for ODK").

Does it succeed now?

Yes.

* Logic: Covered by Zero-Trust.

* History: Covered by persistence_schema.

* Usability: Covered by UX_Constraints.

* Platform: Covered by Platform_Config.

This is now a complete, end-to-end engineering plan that accounts for both the theoretical correctness and the practical deployment challenges.

Red team from Gemini deep research 260216

The digitization of clinical manuals into actionable, decision-supported mobile applications for Community Health Workers represents a highly complex intersection of health informatics, user experience design, and data interoperability. The Community Health Toolkit serves as a foundational, offline-first open-source architecture designed to support such deployments in resource-constrained environments. Concurrently, frameworks like the Gravity Project seek to standardize the capture and interoperability of Social Determinants of Health data—such as housing instability and food insecurity—across diverse digital platforms. The "Project Gravity Gold Master v5.1" plan proposes a methodology for translating narrative clinical guidelines into Community Health Toolkit-compatible XLSForms. While the baseline strategy incorporates modern data modeling concepts, a rigorous architectural critique reveals significant vulnerabilities regarding state persistence, cognitive overload management, and logic execution within the Open Data Kit ecosystem.

This comprehensive research document provides an exhaustive evaluation of the proposed plan. The analysis deconstructs the Two-Table Decision Model and Notation architecture, exposes critical system-level vulnerabilities characterized as "silent killers" within Open Data Kit XPath evaluations, and engineers advanced schemas alongside testing frameworks required to harden the deployment for clinical environments. The objective is to transition the theoretical framework into a highly resilient, interoperable, and cognitively optimized digital health intervention.

1. Architectural Evaluation: The Two-Table DMN Approach

The core challenge in digitizing clinical manuals lies in translating narrative text into deterministic, machine-readable logic without losing clinical fidelity. The proposed Two-Table Decision Model and Notation architecture attempts to resolve this challenge by enforcing a strict separation between navigational routing and clinical logic.

1.1 Structural Efficacy of the Two-Table Design

In complex clinical algorithms, combining workflow navigation with medical threshold evaluation inherently leads to fragile, monolithic form designs. The Two-Table Decision Model and Notation approach mitigates this by enforcing a structural separation of concerns. The first table governs the state and routing logic, dictating which modules or groups of questions are rendered based on prior inputs and the specific role of the health worker. The second table isolates the clinical

decision support logic, housing the physiological and diagnostic thresholds, such as identifying severe pneumonia based on age-adjusted respiratory rates and the presence of lower chest wall indrawing.

This bifurcation strategy closely aligns with the World Health Organization's SMART (Standards-based, Machine-readable, Adaptive, Requirements-based, and Testable) Guidelines initiative, specifically the formulation of Digital Adaptation Kits. Digital Adaptation Kits emphasize the separation of business process workflows from core data dictionaries and decision-support logic. By utilizing a spreadsheet format to explicitly define inputs, outputs, and triggers for each decision logic sequence, the Two-Table architecture ensures that clinical thresholds can be reviewed, verified, and validated by medical subject matter experts independently of the software's syntactic navigation rules. The utilization of Decision Model and Notation standardizes the representation of these rules, enabling automated verification using Satisfiability Modulo Theories solvers like Z3 to mathematically prove that the logical matrix contains no dead ends or contradictory clinical directives.

1.2 Identified Gaps in the DMN to XLSForm Translation

Despite the theoretical soundness of the Two-Table approach, the practical translation of this architecture into the Open Data Kit XForms standard via the XLSForm format presents severe structural limitations. Decision Model and Notation operates on stateless, functional paradigms where given a specific set of inputs, the matrix always produces identical outputs. However, Community Health Worker workflows are inherently stateful, longitudinal, and deeply contextual.

The Gold Master v5.1 plan lacks a sophisticated mechanism to handle missing data and historical patient context. When a Decision Model and Notation table is compiled into an XLSForm using the `calculate` and `relevant` logic columns, the resulting XPath expressions operate under the assumption that all required data points are present within the current session. If a Community Health Worker evaluates a patient whose baseline physiological data or Social Determinants of Health screening was captured in a previous encounter, the isolated decision logic will fail unless it is explicitly engineered to query the application's underlying database for historical markers.

Furthermore, static decision tables struggle to articulate complex temporal constraints, such as scheduling logic for follow-up visits based on the shifting duration of symptoms, which require dynamic date-time calculations integrated seamlessly into the presentation layer. To optimize this architecture, the plan must evolve from a static Two-Table matrix into a dynamic State Machine pattern. Utilizing bitmasking techniques within XLSForm calculations allows the system to evaluate multiple concurrent clinical states efficiently, updating the patient's risk profile dynamically as new variables are introduced, rather than relying on sequential, heavily nested conditional statements that bloat the XML payload and degrade mobile device performance.

2. Red Team Vulnerability Analysis: The "Silent Killers"

Deploying clinical decision support algorithms in low-resource environments introduces unique operational failure modes. Red teaming the Gold Master v5.1 architecture exposes several "silent killers"—systemic, underlying flaws that do not precipitate outright application crashes but instead silently corrupt data collection, misguide clinical interventions, and ultimately degrade patient outcomes.

2.1 The XPath Boolean Trap and Missingness Behavior

One of the most pervasive vulnerabilities in systems relying on the Open Data Kit specification is the divergent evaluation of boolean values between different XForm engines, specifically the differences between mobile clients like ODK Collect and web-based clients like Enketo. In the underlying XML structure, a node storing a boolean result is serialized as text. Under the strict specifications of W3C XPath 1.0, the `boolean()` evaluation function processes any non-empty string as a mathematically true statement.

If the Two-Table architecture compiles a clinical rule into an XLSForm `relevant` column relying on an implicit boolean evaluation—for instance, checking if a previous calculation yielded the string "false" to skip an irrelevant screening module—the web client will interpret the string "false" as `true()`. This results in the presentation of unnecessary clinical modules, confusing the user and corrupting the workflow. The mitigation protocol requires the architecture to enforce strict data casting rules within the XLSForm standard. Developers must completely abandon implicit boolean evaluations and instead utilize explicit string comparisons, or preferably, numeric casting. By encapsulating boolean expressions within the `number()` function, the result is stored securely as a binary integer, which backend relational databases and analytics pipelines correctly interpret without syntactic ambiguity.

Additionally, XPath processing handles missing data unpredictably. Complex clinical algorithms frequently require the calculation of aggregate risk scores. If a decision table evaluates a mathematical equation where one variable was skipped or remains uncollected, the entire expression may yield a `NaN` (Not a Number) output. This missingness behavior silently propagates downstream, paralyzing subsequent logic gates and preventing the triggering of life-saving clinical alerts. The architecture must heavily utilize the `coalesce()` function within XLSForm calculations to assign safe default values to unanswered questions before they enter mathematical operators.

2.2 Circular Dependencies and Directed Acyclic Graphs

The compilation of complex clinical workflows into XLSForms requires the creation of a Directed Acyclic Graph for calculation logic. A critical failure mode occurs when calculation fields inadvertently reference themselves or create a dependency cycle. While validation tools will reject forms where a calculation statement creates a direct, endless loop, highly complex forms spanning hundreds of variables often obscure these cycles through intermediary references until the application is deployed. When these cyclical dependencies execute on low-memory mobile devices, they trigger infinite recalculation loops that drain battery life, freeze the user interface, and destroy unsaved clinical data.

2.3 Alert Fatigue and Automation Bias

A significant clinical risk in implementing decision support systems is the generation of irrelevant, inaccurate, or excessively frequent alerts. Studies across health informatics indicate that clinical decision support malfunctions—such as alerts firing for the wrong demographic due to data dictionary mismatches, or guidelines becoming decoupled during software updates—are widespread and significantly contribute to alert fatigue. When health workers are bombarded with non-critical warnings, they develop a psychological habituation that causes them to ignore truly critical danger signs.

Conversely, automation bias presents an equally dangerous paradigm. When the digital application is perceived as an infallible diagnostic authority, Community Health Workers may over-rely on the software's recommendations, accepting the algorithm's output without independently verifying the clinical context or performing necessary physical examinations. The

architecture must implement context-aware suppression algorithms and ensure that the application functions as an assistive tool rather than a diagnostic replacement. The user interface must transparently present the rationale for its recommendations, allowing the human operator to understand the data points that triggered the specific decision support pathway.

3. Resolving State Persistence and Cognitive Overload

Community Health Workers operate under immense cognitive strain, balancing nuanced patient interactions, environmental distractions, and complex digital data entry. The Gold Master v5.1 plan must deliberately engineer the user experience and the underlying data architecture to minimize this burden while ensuring the longitudinal integrity of the patient's medical record.

3.1 Cognitive Load Theory in mHealth Design

Evaluations of mobile health applications using the Goals, Operators, Methods, and Selection rules (GOMS) cognitive task analysis method reveal that mental operators can account for upwards of 75% of total task fulfillment time. When a digital application forces a health worker to manually synthesize historical patient data, interpret convoluted screening questions, and navigate disorienting user interfaces, cognitive overload occurs. This psychological state directly correlates to increased data entry errors, protocol non-compliance, and the ultimate abandonment of the digital tool.

To optimize the deployment, the architecture must systematically reduce the volume of required mental operators through strict adherence to user-centered design principles. Progressive disclosure is paramount; related variables should be grouped logically and presented on a single screen using the `field-list` appearance attribute, thereby reducing the physical and cognitive friction of continuous screen switching. Furthermore, the ambiguity of required versus optional fields must be eliminated. Users frequently misinterpret the standard asterisk symbol. Cognitive load is significantly reduced by explicitly defining mandatory fields at the outset of the application or clearly denoting optional fields to streamline the perceived workload.

3.2 State Persistence and Longitudinal Data Hydration

A primary driver of cognitive overload in fragmented digital health systems is the necessity for redundant data entry. If a health worker must re-evaluate chronic conditions, demographic baselines, or established Social Determinants of Health during every encounter, the system fails to act as a longitudinal record.

The Community Health Toolkit ecosystem resolves this architectural gap via its robust state persistence model, utilizing an offline-first CouchDB NoSQL database framework. The `medic` database serves as the primary, highly secure repository for all contact profiles and submitted reports. To persist state across multiple encounters, the architecture leverages the `contact-summary.templated.js` script. This script operates as the critical interface between the local CouchDB data store and the client-side user interface, injecting historical patient context directly into the current session.

When a health worker initiates a new clinical assessment, the system dynamically queries the database and retrieves an array of previous reports—loading up to 500 of the latest submissions—alongside the contact's hierarchical lineage, including their parent facility and demographic profile. This hydration of the form state enables the XLSForm to pull persistent variables, such as pre-existing Gravity Project Social Determinants of Health risk factors, directly into the form's logic layer. Consequently, the decision support algorithms can calculate

longitudinal trends, such as growth faltering in pediatric patients or chronic hypertension management, without demanding redundant manual input from the operator.

4. Comprehensive Schema Specifications

To guarantee system stability, semantic interoperability, and rigorous compliance with global health standards, the architecture requires exhaustively defined data schemas. These schemas govern how data is structured, stored, and transmitted, ensuring that the Community Health Toolkit deployment can seamlessly integrate with national health information systems, such as DHIS2, and standardized data repositories.

4.1 Patient Data and Contact Schema

Within the Community Health Toolkit architecture, patient records, health workers, and physical health facilities are universally treated as "contacts" and stored as JSON document objects within CouchDB. The schema dictates the precise structure of demographic, clinical, and Social Determinants of Health data. To maintain strict alignment with the Gravity Project's mandate for integrating social risk data into clinical care, these entities are designed to map to HL7 Fast Healthcare Interoperability Resources (FHIR) standards.

Database Property	Data Type	Structural Constraint	Architectural Description and Purpose
`_id`	UUID	Strictly Required	CouchDB's universally unique identifier, serving as the primary key for the JSON document.
`_rev`	String	Strictly Required	CouchDB's internal revision marker, critical for managing offline synchronization conflicts.
`type`	String	Strictly Required	Denotes the fundamental document type, distinguishing between `person`, `clinic`, or `health_center`.
`contact_type`	String	Environment Conditional	Specifies the custom configured hierarchy type for deployments utilizing CHT versions 3.7 and above.
`reported_date`	Timestamp	Strictly Required	The numerical epoch timestamp indicating the exact millisecond of document creation.

patient_id	String	Operationally Optional	An external, standardized identification number utilized for cross-referencing with national registries.
date_of_birth	Date Format	Clinically Required	The foundational variable required for dynamic age calculation, essential for age-dependent clinical algorithms.
parent	JSON Object	Strictly Required	Defines the geographic and administrative lineage, linking a patient to a specific household or supervisor.

4.2 Comprehensive Audit Log and Telemetry Schema

Robust, immutable auditing is both legally and operationally mandatory for clinical decision support systems. Audit trails allow administrators to reconstruct adverse clinical events, monitor user engagement, and ensure compliance with patient privacy regulations. The enhanced architecture implements a dual-layer logging strategy: client-side telemetry captured via Open Data Kit XForms, and server-side tracking via the CHT medic-audit database.

Client-Side ODK Audit Log Schema: Requested via the orx:audit metadata element embedded within the XLSForm settings, the mobile client generates a structured CSV file tracking granular user behaviors, interaction durations, and form navigation patterns.

CSV Column Header	Logged Data Description and Telemetry Purpose
event	The explicit identifier for the tracked action, such as a question prompt, a background calculation, or a form save operation.
node	The precise XPath reference pointing to the specific question or logic group that triggered the recorded event.
start	The millisecond epoch timestamp denoting the exact beginning of the user interaction.
end	The millisecond epoch timestamp denoting the completion of the interaction, utilized for cognitive load and time-in-app analysis.
old-value	If the form definition configures change tracking, this records the data value prior to the user's modification.
new-value	If change tracking is active, this records the updated data value following the user's input, vital for detecting data tampering.

user	The non-blank identity metric of the enumerator, crucial for maintaining accountability in multi-user device workflows.

Server-Side medic-audit Schema: Simultaneously, the Community Health Toolkit backend maintains a chronological record of all document modifications. To prevent database bloat and ensure optimal query performance, the system rotates audit documents at a maximum of 10 history entries. When a document is modified beyond this limit, the existing entries are archived into a new rotated audit document utilizing an _id format of <document_uuid>:<last rev in history entry>. This ensures that all clinical decisions and state mutations are indefinitely preserved for red-team analysis and programmatic evaluation.

4.3 Phrase Bank, Fact Sheet, and Terminology Schema

To eliminate linguistic ambiguity, support rapid localization, and ensure the semantic interoperability of the collected data, the architecture deploys a highly standardized terminology schema. This schema relies extensively on the World Health Organization's SMART Guidelines Core Data Dictionary structure, which systematically maps clinical inputs and outputs to established international classification codes.

Core Clinical Concept	Terminology System	Standardized Code	Clinical Context and Implementation Rationale
Food Insecurity Risk	LOINC	88122-7	Maps the Gravity Project screening question: "Within the past 12 months, we worried whether our food would run out".
Food Insecurity Diagnosis	ICD-10-CM	Z59.41	Diagnosis mapping for health system interoperability and advanced population health analytics.
Social Care Referral	SNOMED CT	464131000124100	Procedure mapping for initiating a formal intervention via a Community Health Worker.
Fast Breathing Indicator	ICD-11	MD11	A primary clinical indicator utilized within the IMCI algorithm to trigger a suspected pneumonia workflow.
Severe Chest Indrawing	SNOMED CT	248568003	Danger sign mapping utilized to mandate an immediate referral to a higher-tier medical facility.

The phrase bank operates as the single source of truth for the XLSForm's label and hint columns. By decoupling the display text from the underlying logic, the architecture supports automated, large-scale translation workflows, allowing the application to be deployed across diverse linguistic regions without altering the core XML logic. The Fact Sheet schema, derived directly from the narrative clinical manual, provides the structured health education messages

and protocol summaries that are rendered to the user upon the completion of a diagnostic module.

4.4 UX Constraints and Interface Schema

The presentation layer of the XLSForm directly impacts the human operator's ability to process critical medical information rapidly and accurately. The User Experience constraint schema defines strict, immutable rendering rules to maintain visual consistency and reduce cognitive friction across the entire application suite.

User Interface Element	Styling Rule / Constraint	Cognitive Purpose and Clinical Application
Patient Details Header	Yellow (#e2b100)	Establishes the patient's identity and demographic context immediately upon opening a form, preventing wrong-patient errors.
Standard Visit Info	Blue (#6b9acd)	Serves as the default, low-stress fallback color for routine assessment questions, vital signs, and standard instructions.
Warning/Danger Signs	Red (#e00900)	Aggressively highlights critical clinical thresholds that mandate an immediate emergency referral or intervention.
Follow-up Directives	Green (#75b2b2)	Indicates the safe resolution of the current clinical workflow and clearly dictates the next steps for the health worker.
Input Value Constraints	Dynamic (regex)	Prevents mathematically impossible entries at the point of care, such as restricting body temperature inputs to plausible human limits.

6. Red Teaming the Revised Architecture

To continuously secure the "Gold Master v5.1" architecture, an ongoing, aggressive Red Team protocol must be established. This protocol is designed to intentionally stress-test the environment against real-world degradation, ensuring that the system fails safely rather than catastrophically.

6.1 Simulating Data Synchronization and Network Failures

The Community Health Toolkit architecture relies heavily on the medic-sentinel database to process state changes sequentially and manage the flow of data between the offline device and the central server. Red team simulations must intentionally throttle and sever network connectivity to observe how the application handles delayed replication. If a health worker records a severe clinical danger sign while entirely offline, the system must be rigorously validated to ensure that the local device strictly enforces the clinical referral protocol and generates the necessary emergency alerts without requiring any server-side validation or

round-trip logic execution. The system must cache these critical events flawlessly, pushing them to the server the moment connectivity is restored without data loss or sequence corruption.

6.2 Analytics and Database Desynchronization Attacks

For health ministries, program managers, and stakeholders, the collected data is visualized using robust PostgreSQL pipelines fed by the CHT Sync utility. Red team analysts must inject malformed, highly recursive, or syntactically corrupt datasets directly into the CouchDB instance to test the resilience of the Data Build Tool (dbt) transformations that bridge the NoSQL and SQL environments. The architectural schema must cleanly reject anomalous geographic coordinates, corrupted date-time stamps, or severe schema deviations without halting or crashing the broader analytics synchronization pipeline.

6.3 Algorithmic Drift and Versioning Conflicts

As clinical guidelines—such as the WHO Integrated Management of Childhood Illness (IMCI) protocols—inevitably evolve over time, the underlying clinical decision support algorithms will require remote updates. The architecture must strictly enforce version control within the XLSForm settings sheet utilizing the `version` attribute. A critical operational failure mode exists where updated clinical logic deployed to the central server conflicts with cached, outdated forms currently active on a health worker's offline device. Red teaming must aggressively validate the conflict-resolution matrices during the synchronization process, ensuring that backward compatibility is maintained for records created under the old logic, and that deprecated logic does not maliciously overwrite or corrupt newer clinical guidelines upon device reconnection.

7. Strategic Conclusions

The translation of the "Project Gravity Gold Master v5.1" from a conceptual Two-Table Decision Model and Notation plan into a highly robust, deployment-ready Community Health Toolkit architecture requires meticulous, defensive software engineering. The theoretical separation of routing and clinical logic must be functionally realized through dynamic bitmask state machines that respect the limitations of mobile processing environments.

By identifying and nullifying the Open Data Kit XPath boolean traps, restructuring static decision matrices into state-persistent mechanisms driven by CouchDB and the `contact-summary.templated.js` hydration process, and fundamentally optimizing the user interface to mitigate the severe cognitive load placed on frontline health workers, the system guarantees a significantly higher degree of clinical fidelity.

The comprehensive implementation of strictly defined, immutable schemas mapped directly to HL7 FHIR terminologies ensures seamless data interoperability and standardizes the capture of critical Social Determinants of Health. Furthermore, the aggressive deployment of Mocha and Chai integration harnesses, alongside automated XForm unit testing, mathematically validates the integrity of every clinical decision pathway. This exhaustively engineered architecture secures the deployment at every layer, successfully transforming static clinical manuals into dynamic, deeply resilient, and life-saving digital tools operating at the very edge of the global healthcare system.

Z3 prover & patient generator

Z3 solver

This document includes:

1. Python to run Z3 for any DMN tables (e.g., exhaustive)
2. A prompt to generate Z3 for specific DMN tables
3. Generating synthetic patients
4. running simulated patients
5. Red team critiques of parts 1-3

Z3 for any DMN tables

```
This is draft code for any set of DMN tables.

# Feb 17, 2026
# Evergreen DMN Verification Tests (Z3 + simple graph checks)
#
# PURPOSE
#   This file contains a generic test suite you can run on *any* DMN
package
#  It checks evergreen safety/quality properties:
#     - domain is satisfiable (non-vacuous)
#     - every rule is reachable (no dead rules)
#     - every table is exhaustive (no gaps), under the declared
DOMAIN
#     - overlap diagnostics (pairwise overlaps) for every table
#     - policy-specific checks:
#         UNIQUE: MECE (mutually exclusive + collectively exhaustive)
#         FIRST/PRIORITY: shadowing + ordered exhaustiveness
#     - router checks:
#         (Level 1) static potential-cycle detection on module
transition graph
#         (Level 2) bounded SMT cycle search (requires a
RouterSemantics adapter)
#
# SETUP
#   pip install z3-solver
#
# USAGE (minimal)
#   1) Build DMNTable objects with Z3 antecedents (BoolRef) for each
rule.
#   2) Provide a DOMAIN constraint (BoolRef) describing feasible
boolean inputs.
```

```
#      IMPORTANT: if booleans are derived from raw values, DOMAIN
should include
#      the equivalences or you will get false positives/negatives.
#   3) Call:
#        report = verify_package(pkg)
#        report.raise_if_failed()
#
# DESIGN PHILOSOPHY
#   - Python orchestrates and reports; Z3 only solves.
#   - Every test produces either:
#       PASS, or FAIL with a concrete counterexample model where
possible.
#   - "Exhaustive" and "MECE" are always interpreted relative to
DOMAIN.
#     DOMAIN is part of the spec. If DOMAIN is too weak, proofs are
weaker.
#
# LIMITATIONS / TODO
#   - Router bounded-cycle search requires a formal state update
semantics.
#     This file provides an interface (RouterSemantics) and a
skeleton test.
#   - Output-typing tests are included but depend on how you encode
outputs.
#     If outputs are strings, you'll likely map them to Enums/Int
codes.

from __future__ import annotations

from dataclasses import dataclass, field
from itertools import combinations
from typing import Dict, List, Optional, Tuple, Any, Set

from z3 import (
    BoolRef, Solver, sat, unsat, And, Or, Not,
    ModelRef, is_true
)

# ----------------------------
# Data structures (DMN package)
# ----------------------------

@dataclass(frozen=True)
class DMNRule:
    """
```

```
    A single DMN rule:
      - rule_id: stable identifier
      - when: Z3 BoolRef representing the rule antecedent
      - outputs: dictionary of outputs (optional; used for output
checks)
    """
    rule_id: str
    when: BoolRef
    outputs: Dict[str, Any] = field(default_factory=dict)


@dataclass(frozen=True)
class DMNTable:
    """
    A DMN decision table.
      - table_id: stable identifier
      - hit_policy: 'COLLECT', 'UNIQUE', 'FIRST', or 'PRIORITY'
      - rules: ordered list of DMNRule; order matters for
FIRST/PRIORITY
      - kind: optional semantic label, e.g. 'ACTIVATOR', 'ROUTER',
'CLASSIFIER'
      - meta: arbitrary metadata (e.g., module names, current module
symbol)
    """
    table_id: str
    hit_policy: str
    rules: List[DMNRule]
    kind: str = "GENERIC"
    meta: Dict[str, Any] = field(default_factory=dict)


@dataclass(frozen=True)
class DMNPackage:
    """
    The full DMN “release artifact” you want to verify.
      - domain: Z3 BoolRef constraint limiting feasible inputs
      - tables: list of decision tables
      - router_table_ids: which tables are "traffic cops" (routing)
    """
    domain: BoolRef
    tables: List[DMNTable]
    router_table_ids: Set[str] = field(default_factory=set)


# ----------------------------
```

```
# Reporting / results
# ----------------------------

@dataclass
class TestFailure:
    test_id: str
    table_id: Optional[str]
    message: str
    witness_model: Optional[Dict[str, Any]] = None


@dataclass
class VerificationReport:
    failures: List[TestFailure] = field(default_factory=list)
    warnings: List[str] = field(default_factory=list)

    def ok(self) -> bool:
        return len(self.failures) == 0

    def add_failure(self, test_id: str, table_id: Optional[str],
message: str,
                    witness_model: Optional[Dict[str, Any]] = None)
-> None:
        self.failures.append(TestFailure(test_id, table_id, message,
witness_model))

    def add_warning(self, message: str) -> None:
        self.warnings.append(message)

    def raise_if_failed(self) -> None:
        if not self.ok():
            lines = ["DMN verification FAILED:"]
            for f in self.failures:
                lines.append(f"- [{f.test_id}] table={f.table_id or
'ALL'}: {f.message}")
                if f.witness_model:
                    lines.append(f"  witness={f.witness_model}")
            if self.warnings:
                lines.append("Warnings:")
                for w in self.warnings:
                    lines.append(f"- {w}")
            raise AssertionError("\n".join(lines))


# ----------------------------
```

```
# Z3 helper functions
# ----------------------------

def _solve_one(constraints: List[BoolRef]) -> Tuple[bool,
Optional[ModelRef]]:
    """
    Solve constraints once.
      Returns (is_sat, model_if_sat).
    """
    s = Solver()
    for c in constraints:
        s.add(c)
    res = s.check()
    if res == sat:
        return True, s.model()
    return False, None


def _model_to_dict(model: ModelRef, only_these:
Optional[List[BoolRef]] = None) -> Dict[str, Any]:
    """
    Convert a Z3 model to a dict of variable->value strings.

    NOTE:
      Z3 models include only symbols Z3 decides to show;
model_completion=True can be used
      if you know the variable objects. Here we keep it generic for
readability:
      - We list all model declarations by default.
    """
    out: Dict[str, Any] = {}
    # Z3 model: iterate through declared symbols
    for d in model.decls():
        out[str(d.name())] = str(model[d])
    return out


# ----------------------------
# Evergreen tests: domain & reachability & exhaustiveness
# ----------------------------

def test_domain_satisfiable(pkg: DMNPackage, report:
VerificationReport) -> None:
    """
    Test 0: DOMAIN must be SAT (otherwise everything is vacuous).
```

```
    If DOMAIN is UNSAT, you either:
      - wrote contradictory feasibility constraints, or
      - forgot to include needed degrees of freedom,
      - or used inconsistent derived-feature equivalences.
    """
    ok, m = _solve_one([pkg.domain])
    if not ok:
        report.add_failure(
            test_id="T00_DOMAIN_SAT",
            table_id=None,
            message="DOMAIN is UNSAT (vacuous spec). Fix
domain/feature constraints.",
            witness_model=None,
        )


def test_rule_reachability(pkg: DMNPackage, report:
VerificationReport) -> None:
    """
    Test 2: For each rule r, check DOMAIN ∧ r.when is SAT.

    If UNSAT, rule is unreachable / dead code.
    """
    for t in pkg.tables:
        for r in t.rules:
            ok, m = _solve_one([pkg.domain, r.when])
            if not ok:
                report.add_failure(
                    test_id="T02_RULE_REACHABLE",
                    table_id=t.table_id,
                    message=f"Rule {r.rule_id} unreachable: DOMAIN ∧
when is UNSAT.",
                    witness_model=None,
                )


def test_table_has_any_match(pkg: DMNPackage, report:
VerificationReport) -> None:
    """
    Test 1: For each table, check DOMAIN ∧ (∨ rule.when) is SAT.

    If UNSAT, table can never match any input state.
    """
    for t in pkg.tables:
```

```
        if not t.rules:
            report.add_failure(
                test_id="T01_TABLE_NONEMPTY",
                table_id=t.table_id,
                message="Table has zero rules.",
                witness_model=None,
            )
            continue

        any_match = Or(*[r.when for r in t.rules])
        ok, m = _solve_one([pkg.domain, any_match])
        if not ok:
            report.add_failure(
                test_id="T01_TABLE_SOME_MATCH",
                table_id=t.table_id,
                message="Table never matches: DOMAIN ∧ (OR all
rules) is UNSAT.",
                witness_model=None,
            )


def test_table_exhaustive(pkg: DMNPackage, report:
VerificationReport) -> None:
    """
    Test 3: For each table, check exhaustiveness under DOMAIN:
        DOMAIN ∧ ¬(∨ rule.when) must be UNSAT.

    If SAT, Z3 provides a counterexample input that matches no rule.
    """
    for t in pkg.tables:
        any_match = Or(*[r.when for r in t.rules]) if t.rules else
False
        ok, m = _solve_one([pkg.domain, Not(any_match)])
        if ok:
            report.add_failure(
                test_id="T03_TABLE_EXHAUSTIVE",
                table_id=t.table_id,
                message="Table is NOT exhaustive: found input state
where no rule matches.",
                witness_model=_model_to_dict(m) if m else None,
            )


def overlap_diagnostics(pkg: DMNPackage, report: VerificationReport)
-> Dict[str, List[Tuple[str, str, Dict[str, Any]]]]:
```

```
    """
    Test 6 (diagnostic): For each table, compute overlaps:
      For each pair (ri,rj), check DOMAIN ∧ Ai ∧ Aj is SAT.
      If SAT, store witness as a diagnostic.

    Overlap may be allowed (COLLECT), forbidden (UNIQUE), or handled
by order (FIRST).
    We store these overlaps for later policy-specific logic and human
review.
    """
    overlaps: Dict[str, List[Tuple[str, str, Dict[str, Any]]]] = {}

    for t in pkg.tables:
        hits: List[Tuple[str, str, Dict[str, Any]]] = []
        for r1, r2 in combinations(t.rules, 2):
            ok, m = _solve_one([pkg.domain, r1.when, r2.when])
            if ok and m is not None:
                hits.append((r1.rule_id, r2.rule_id,
_model_to_dict(m)))
        overlaps[t.table_id] = hits

        # Add a warning if there are overlaps, even if not
necessarily invalid.
        if hits:
            report.add_warning(
                f"Overlap diagnostics: table {t.table_id} has
{len(hits)} overlapping rule pairs "
                f"(not necessarily wrong; depends on hit policy)."
            )
    return overlaps


# ----------------------------
# Policy-specific evergreen tests
# ----------------------------

def test_unique_mece(pkg: DMNPackage, report: VerificationReport) ->
None:
    """
    For tables with hit_policy == UNIQUE:
      - Mutual exclusivity: for all pairs i≠j, DOMAIN ∧ Ai ∧ Aj
must be UNSAT
      - Exhaustiveness: already covered by T03, but we repeat with a
UNIQUE tag
    """
```

```
    for t in pkg.tables:
        if t.hit_policy.upper() != "UNIQUE":
            continue

        # Mutual exclusivity
        for r1, r2 in combinations(t.rules, 2):
            ok, m = _solve_one([pkg.domain, r1.when, r2.when])
            if ok:
                report.add_failure(
                    test_id="T_UNIQUE_ME_PAIRWISE_UNSAT",
                    table_id=t.table_id,
                    message=f"UNIQUE violates mutual exclusivity:
{r1.rule_id} AND {r2.rule_id} can both be true.",
                    witness_model=_model_to_dict(m) if m else None,
                )

        # Collectively exhaustive (same as T03 but tagged)
        any_match = Or(*[r.when for r in t.rules])
        ok, m = _solve_one([pkg.domain, Not(any_match)])
        if ok:
            report.add_failure(
                test_id="T_UNIQUE_ME_EXHAUSTIVE",
                table_id=t.table_id,
                message="UNIQUE violates exhaustiveness: some
feasible input matches no rule.",
                witness_model=_model_to_dict(m) if m else None,
            )


def test_first_priority_shadowing(pkg: DMNPackage, report:
VerificationReport) -> None:
    """
    For tables with hit_policy in {FIRST, PRIORITY}:
      Let rules be ordered r1, r2, ..., rn.
      Define "fires_k" = A_k ∧ ¬(A_1 ∨ ... ∨ A_{k-1})

    Evergreen checks:
      1) Each fires_k is SAT (otherwise rule is completely shadowed
by earlier rules)
      2) Ordered exhaustiveness: DOMAIN ∧ ¬(fires_1 ∨ ... ∨
fires_n) must be UNSAT
    """
    for t in pkg.tables:
        if t.hit_policy.upper() not in {"FIRST", "PRIORITY"}:
            continue
```

```
        if not t.rules:
            continue

        fires: List[BoolRef] = []
        prev_or = None

        for idx, r in enumerate(t.rules):
            if idx == 0:
                fire = r.when
                prev_or = r.when
            else:
                fire = And(r.when, Not(prev_or))
                prev_or = Or(prev_or, r.when)

            fires.append(fire)

            # Shadowing check: DOMAIN ∧ fires_k must be SAT
            ok, m = _solve_one([pkg.domain, fire])
            if not ok:
                report.add_failure(
                    test_id="T_FIRST_SHADOWED_RULE",
                    table_id=t.table_id,
                    message=f"Rule {r.rule_id} is shadowed: it can
never be selected under FIRST/PRIORITY order.",
                    witness_model=None,
                )

        # Ordered exhaustiveness
        ok, m = _solve_one([pkg.domain, Not(Or(*fires))])
        if ok:
            report.add_failure(
                test_id="T_FIRST_ORDERED_EXHAUSTIVE",
                table_id=t.table_id,
                message="FIRST/PRIORITY violates exhaustiveness:
feasible input selects no rule under ordering.",
                witness_model=_model_to_dict(m) if m else None,
            )


# ----------------------------
# Output well-formedness checks (generic hooks)
# ----------------------------

def test_outputs_in_allowed_sets(pkg: DMNPackage, report:
VerificationReport,
```

```
                                allowed_outputs: Dict[str,
Set[Any]]) -> None:
    """
    Test 5 (generic): ensure each rule's output values are in allowed
sets.

    Example:
      allowed_outputs = {
        "next_module": {"fever", "diarrhea", "DONE", "REFER"},
        "action": {"TREAT", "REFER", "ADVISE"},
      }

    This is a meta-check (not SMT) unless you model outputs as Z3
enums/int vars.
    Still valuable because it catches typos from LLMs (e.g.,
"diarrhoea" vs "diarrhea").
    """
    for t in pkg.tables:
        for r in t.rules:
            for k, v in r.outputs.items():
                if k not in allowed_outputs:
                    continue  # no restriction for this output key
                if v not in allowed_outputs[k]:
                    report.add_failure(
                        test_id="T05_OUTPUT_ALLOWED",
                        table_id=t.table_id,
                        message=f"Rule {r.rule_id} output {k}={v} not
in allowed set.",
                        witness_model=None,
                    )


# ----------------------------
# Router checks
# ----------------------------

def build_router_graph(pkg: DMNPackage) -> Dict[str, Set[str]]:
    """
    Router graph (Level 1): build a directed graph module_i ->
module_j based on
    routing rules that output next_module=j.

    This is intentionally conservative: it ignores rule conditions
and state updates.
    A cycle here is a *warning* (potential loop), not a proof.
```

```
    Requires each router rule to have outputs["from_module"] and
outputs["next_module"].
    If your representation differs, adapt this function.
    """
    g: Dict[str, Set[str]] = {}

    for t in pkg.tables:
        if t.table_id not in pkg.router_table_ids:
            continue

        for r in t.rules:
            frm = r.outputs.get("from_module")
            nxt = r.outputs.get("next_module")
            if frm is None or nxt is None:
                continue
            g.setdefault(frm, set()).add(nxt)

    return g


def find_cycles_in_graph(g: Dict[str, Set[str]]) -> List[List[str]]:
    """
    Standard DFS cycle detection returning example cycles.
    This is NOT SMT; it’s a fast structural check.
    """
    cycles: List[List[str]] = []
    visiting: Set[str] = set()
    visited: Set[str] = set()
    stack: List[str] = []

    def dfs(u: str):
        visiting.add(u)
        stack.append(u)

        for v in g.get(u, set()):
            if v in visiting:
                # Found a back-edge; extract cycle from stack
                i = stack.index(v)
                cycles.append(stack[i:] + [v])
            elif v not in visited:
                dfs(v)

        stack.pop()
        visiting.remove(u)
```

```
        visited.add(u)

    for node in list(g.keys()):
        if node not in visited:
            dfs(node)

    return cycles


class RouterSemantics:
    """
    Interface for SMT-based router loop checking (Level 2).
    You must implement how the router updates state between steps.

    Why this is needed:
      A router loop depends on state updates (queue, completed list,
referral).
      Pure DMN antecedents alone cannot prove termination.

    Required methods:
      - declare_state(step): return dict of Z3 vars for that step
      - transition(step, state_s, state_s1): return BoolRef encoding
one router step
      - domain_state(state): feasibility constraints for router state
      - is_terminal(state): BoolRef for DONE/REFER terminal
conditions (so loops exclude terminals)
    """
    def declare_state(self, step: int) -> Dict[str, Any]:
        raise NotImplementedError

    def transition(self, step: int, s: Dict[str, Any], s1: Dict[str,
Any]) -> BoolRef:
        raise NotImplementedError

    def domain_state(self, s: Dict[str, Any]) -> BoolRef:
        raise NotImplementedError

    def is_terminal(self, s: Dict[str, Any]) -> BoolRef:
        raise NotImplementedError


def test_router_bounded_cycle(pkg: DMNPackage,
                              report: VerificationReport,
                              semantics: RouterSemantics,
                              k: int = 8) -> None:
```

```
    """
    Router SMT check (Level 2): bounded cycle existence.

    We ask Z3:
      "Is there a feasible initial state s0 and a sequence of k
router transitions
       returning to the same current_module (or same full state),
without hitting terminal?"

    If SAT: you found a concrete loop witness (bad).
    If UNSAT up to k ~ (#modules + 1), you have strong evidence of
termination.

    IMPORTANT:
      This requires a good semantics implementation. Without it, skip
this test.
    """
    # Build step states
    states = [semantics.declare_state(i) for i in range(k + 1)]

    constraints: List[BoolRef] = []

    # State feasibility for each step
    for s in states:
        constraints.append(semantics.domain_state(s))

    # Transitions
    for i in range(k):
        constraints.append(semantics.transition(i, states[i],
states[i + 1]))

    # Exclude terminal states along the way (otherwise cycles may
include DONE/REFER trivially)
    for i in range(k + 1):
        constraints.append(Not(semantics.is_terminal(states[i])))

    # Define what it means to be a "cycle".
    # Common choice: same current_module at end as at start.
    # If you prefer stronger: equality of the whole state. Up to you.
    s0 = states[0]
    sk = states[k]
    if "current_module" in s0 and "current_module" in sk:
        constraints.append(sk["current_module"] ==
s0["current_module"])
    else:
```

```
        # Fallback: can’t check cycle if you didn’t define
current_module
        report.add_warning("Router bounded-cycle test skipped:
semantics lacks current_module.")
        return

    ok, m = _solve_one(constraints)
    if ok:
        report.add_failure(
            test_id="T07_ROUTER_BOUNDED_CYCLE",
            table_id=None,
            message=f"Router may loop: found a satisfiable cycle of
length {k}.",
            witness_model=_model_to_dict(m) if m else None,
        )


# ----------------------------
# Main entry point: run evergreen suite
# ----------------------------

def verify_package(pkg: DMNPackage,
                   allowed_outputs: Optional[Dict[str, Set[Any]]] =
None,
                   router_semantics: Optional[RouterSemantics] =
None,
                   router_cycle_bound: int = 0) ->
VerificationReport:
    """
    Run the evergreen test suite.
      - allowed_outputs: optional meta-check for output vocabularies
      - router_semantics + router_cycle_bound>0: enable bounded SMT
router cycle search
    """
    report = VerificationReport()

    # Always: domain satisfiable
    test_domain_satisfiable(pkg, report)
    if not report.ok():
        # If domain is UNSAT, stop early; everything else would be
noisy.
        return report

    # Always: table nonempty + has any match
    test_table_has_any_match(pkg, report)
```

```
    # Always: rule reachability
    test_rule_reachability(pkg, report)

    # Always: table exhaustiveness
    test_table_exhaustive(pkg, report)

    # Always: overlap diagnostics
    _ = overlap_diagnostics(pkg, report)

    # Policy-specific: UNIQUE => MECE
    test_unique_mece(pkg, report)

    # Policy-specific: FIRST/PRIORITY => shadowing + ordered
exhaustiveness
    test_first_priority_shadowing(pkg, report)

    # Outputs: optional vocabulary checks
    if allowed_outputs:
        test_outputs_in_allowed_sets(pkg, report, allowed_outputs)

    # Router Level 1: static cycle warning
    if pkg.router_table_ids:
        g = build_router_graph(pkg)
        cycles = find_cycles_in_graph(g)
        if cycles:
            report.add_warning(f"Router graph has potential cycles
(structural, not SMT proof): {cycles}")

    # Router Level 2: SMT bounded cycle search (optional)
    if router_semantics is not None and router_cycle_bound and
router_cycle_bound > 0:
        test_router_bounded_cycle(pkg, report, router_semantics,
k=router_cycle_bound)

    return report


# ----------------------------
# Example usage (toy)
# ----------------------------
if __name__ == "__main__":
    # This block is an illustrative example. Replace with your DMN
compiler output.
```

```
    from z3 import Bool

    # Example boolean inputs
    has_fever = Bool("has_fever")
    cough = Bool("cough")
    danger = Bool("danger")

    # DOMAIN: allow any combo (weak domain; in real life you should
constrain feasibility)
    DOMAIN = True  # e.g., And(...) if you have feasibility
constraints

    # A UNIQUE table with 2 rules + default else
    t = DMNTable(
        table_id="triage_unique",
        hit_policy="UNIQUE",
        kind="CLASSIFIER",
        rules=[
            DMNRule("R1_REFER", when=danger, outputs={"action":
"REFER"}),
            DMNRule("R2_FEVER", when=And(has_fever, Not(danger)),
outputs={"action": "FEVER_PATH"}),
            DMNRule("R3_DEFAULT", when=And(Not(has_fever),
Not(danger)), outputs={"action": "HOME_CARE"}),
        ],
    )

    pkg = DMNPackage(domain=DOMAIN, tables=[t],
router_table_ids=set())

    allowed = {"action": {"REFER", "FEVER_PATH", "HOME_CARE"}}

    rep = verify_package(pkg, allowed_outputs=allowed)
    print("OK:", rep.ok())
    print("Warnings:", rep.warnings)
    if not rep.ok():
        for f in rep.failures:
            print(f)
        rep.raise_if_failed()
```

Prompt for LLM to generate additional Z3 tests

You are a formal verification engineer. Your job is to propose additional automated checks (SMT/Z3 queries or simple graph checks) for a package of DMN decision tables.

Inputs I will provide:
1) A list of DMN tables. For each table:
- table_id
- hit_policy ∈ {COLLECT, UNIQUE, FIRST, PRIORITY}
- inputs: list of boolean variable names
- rules: each rule has rule_id, antecedent boolean expression (AND/OR/NOT over inputs), and outputs (strings/enums)
- if PRIORITY/FIRST: a rule order is provided (highest to lowest)
2) Domain constraints over inputs (boolean constraints), called DOMAIN.
3) For router tables: a description of state variables and update semantics (current_module, pending_queue, completed_set, referral terminal state, DONE terminal state).
4) Optional: feature-engineering equivalences linking raw vars to booleans (may be omitted; assume DOMAIN already encodes feasible boolean combos).

Your tasks:
A) Propose additional tests beyond this base suite:
- domain SAT
- per-rule reachability: DOMAIN ∧ antecedent
- per-table exhaustiveness: DOMAIN ∧ ¬(∨ antecedents)
- (diagnostic) overlap matrix: DOMAIN ∧ Ai ∧ Aj
- router bounded-cycle search
- referral precedence
B) Policy-specific requirements:
1) If hit_policy is UNIQUE:
- require MECE: mutually exclusive AND collectively exhaustive over DOMAIN.
- Provide Z3 queries:
(i) for each pair i≠j: DOMAIN ∧ Ai ∧ Aj must be UNSAT
(ii) DOMAIN ∧ ¬(A1 ∨ ... ∨ An) must be UNSAT
2) If hit_policy is FIRST/PRIORITY:
- require determinism under order: for each rule r_k, “fires” = A_k ∧ ¬(A_1∨...∨A_{k-1}).
- Provide tests:
(i) reachability of each effective firing condition
(ii) exhaustiveness of ordered selection: DOMAIN ∧ ¬(fires_1 ∨ ... ∨ fires_n) UNSAT
(iii) check that earlier rules do not completely shadow later ones (fires_k SAT for all k)
3) If hit_policy is COLLECT:
- overlap is allowed; instead propose:
(i) “contradiction” checks: find SAT assignments where two outputs conflict (e.g., “treat” and “refer” both produced) if outputs are meant to be exclusive.
(ii) maximum overlap search (optional): ask Z3 to maximize number of simultaneously true antecedents to stress test downstream logic.
C) Router-specific additional tests:
- No self-loop without progress: current_module stays same while state does not change

- Queue monotonicity: completed_set grows, pending_queue shrinks unless new modules added; specify exactly.
- If a rule adds modules, require “no re-adding completed modules” unless explicitly allowed.
- Provide at least one ranking-function-based proof obligation if possible; if not, propose bounded model checking depth and why.

Output format (strict):
1) A numbered list of proposed tests.
2) For each test:
- test_id
- applies_to: (table_id or ALL or ROUTER_ONLY)
- type: SMT_UNSAT | SMT_SAT | GRAPH | META
- query: a single-line pseudo-SMT formula using DOMAIN and rule antecedents (Ai), or a brief graph algorithm description
- expected: SAT or UNSAT (or PASS/FAIL for GRAPH)
- failure_output: what counterexample/witness to print if it fails
3) Do not write code. Do not write prose explanations longer than 2 sentences per test.
4) Include at least:
- 3 tests that catch common clinical safety bugs (e.g., referral precedence violations, contradictory outputs, missing rule gaps)
- 2 tests that catch workflow bugs (router loops, shadowed rules, non-termination)

Notes on how this prompt helps

- It forces the LLM to generate queries you can actually compile.
- It makes policy differences explicit (UNIQUE vs FIRST vs COLLECT).
- It requires “expected SAT/UNSAT” so the harness can assert outcomes.

Two gotchas (so you don’t get false confidence)

1. Exhaustiveness/MECE is only as good as DOMAIN. If DOMAIN doesn’t encode feasibility (e.g., derived booleans from raw), Z3 may flag overlaps/gaps that can’t happen—or miss ones that can.

2. Loop freedom requires modeling state updates. If you only check the module-to-module graph without queue/completed semantics, you’ll miss real cycles or wrongly flag harmless ones.

If you want, I can also provide a compact “Assertions.yaml” schema that your LLM must output alongside each DMN table so your harness automatically knows which of these checks to run (and with what depth for bounded cycle search).

Generating synthetic patients

You are not “writing test cases.”

You are compiling medicine into logic, and then asking mathematics to generate counterexamples.

Here’s how to explain it clearly to interns.

---

1. The Big Picture

Your system has three layers:

Raw patient data → Feature engineering → DMN decision logic
(temp_c, rr, etc.) (has_fever, fast) (Booleans only)

Key points:

- DMN sees only Booleans.
- But reality contains continuous variables.
- Therefore, synthetic patients must satisfy BOTH:
    - raw clinical constraints
    - derived Boolean definitions
    - DMN rule conditions

We don’t hand-write fake patients. We ask Z3 to solve for them.

---

2. What We Are Actually Doing

We translate:

FEEL (limited subset)

has_fever = true if temp_c > 37.7

Into logic:

has_fever ↔ (temp_c > 37.7)

Then translate DMN row:

IF has_fever AND cough THEN pneumonia

Into:

pneumonia_row_cond = has_fever ∧ cough
Then we ask Z3:“Find me a patient where pneumonia_row_cond is true.”

Z3 gives us a complete synthetic patient:

- temp_c
- has_fever
- cough
- etc.

All consistent.

3. What is “Compilation”?

We are building a mini compiler.

Input languages:

- Limited FEEL (threshold expressions)
- Boolean DMN rows

Target language:

- Z3 constraint logic

So we need:

FEEL form	Z3 form
x > 37.7	x > 37.7
A and B	And(A,B)
A or B	Or(A,B)
not A	Not(A)
has_fever = temp_c > 37.7	has_fever == (temp_c > 37.7)

This is not AI.

This is syntax tree translation.

4. The Implementation

Step 1: Define Variables

Define schema:

```
temp_c : Real
rr     : Int
has_fever : Bool
fast_breathing : Bool
```

Add domain constraints:

30 ≤ temp_c ≤ 43
0 ≤ rr ≤ 120

This prevents nonsense patients.

Step 2: Compile Feature Engineering

Each feature becomes an equivalence constraint:

has_fever ↔ (temp_c > 37.7)

fast_breathing ↔ (
  (age_mo < 12 ∧ rr ≥ 50) ∨
  (age_mo ≥ 12 ∧ rr ≥ 40)
)
Rule: Every Boolean must be defined in terms of raw variables or other Booleans.

No floating features allowed.

Step 3: Compile DMN Rows

Each DMN row becomes:

row_i_condition = And(...)

If hit policy = FIRST:

row_i_fires =
  row_i_condition
  ∧ ¬row_1
  ∧ ¬row_2
  ...

If hit policy = COLLECT:

Just row_i_condition.

Step 4: Generate Synthetic Patients

Now solving is easy.

Per row:

Solve:
  domain_constraints
  ∧ feature_constraints
  ∧ row_i_fires

Z3 returns a model.

That is your synthetic patient.

Step 5: Generate Edge Cases

For threshold testing:

We generate three models:

temp_c = 37.6
temp_c = 37.7
temp_c = 37.8

While pinning everything else constant.

This tests:

- operator correctness (>, ≥)
- rounding bugs
- off-by-one errors
- float errors

This is how you catch clinical disasters.

5. Why This Is Powerful

Traditional testing:

- Humans guess test cases.
- Humans miss edge cases.

Constraint testing:

- Mathematics guarantees coverage.
- You can prove every row is reachable.
- You can prove no row is contradictory.
- You can find impossible combinations.

It is closer to formal verification than unit testing.

6. What NOT To Do

1. ❌ Never manually invent synthetic patients.
2. ❌ Never define Boolean features without equivalence constraints.
3. ❌ Never allow raw variables without domain constraints.
4. ❌ Never mix FEEL parsing with DMN parsing — separate modules.

7. Architectural Diagram

```
FEEL (thresholds)
        ↓
Feature Compiler
        ↓
Boolean layer
        ↓
DMN rows
        ↓
DMN Compiler
        ↓
Z3 Constraints
        ↓
Synthetic Patients
        ↓
Differential Testing vs XLSForm
```

This is your zero-trust pipeline.

8. Two Important Warnings

1) Unsatisfiable rows

If Z3 says UNSAT:

- Either the row is logically impossible
- Or domains are too strict
- Or feature definitions conflict

This is a clinical bug detector, not a failure.

2) Combinatorial explosion

Testing all pairs or triples of diagnoses grows fast.

- Prioritize clinically meaningful overlaps
- Use blocking clauses to avoid duplicate models
- Limit search depth

9. Big picture philosophy

We are not generating random patients.

We are constructing mathematical witnesses.

Each synthetic patient proves:
"This rule is reachable."
or
"This boundary behaves correctly."
That is stronger than coverage testing.

> IN SHORT: We translate FEEL + DMN into mathematics, then use a theorem prover to automatically invent clinically consistent patients that exercise every rule and every edge case.

Running simulated patients

To run a concrete synthetic patient through your Z3-encoded DMN logic, you need to evaluate your symbolic Z3 expressions using the patient's fixed values.
Depending on whether your patient is currently a Z3 ModelRef (just generated by the solver) or a standard Python dictionary (loaded from a JSON file), there are three primary ways to execute this in the Z3 Python API.
1. Using model.evaluate() (If Z3 just generated the patient)
When you ask Z3 to generate a synthetic patient, it returns a Model (a satisfying assignment of variables). You can directly ask this model to evaluate any other rule or expression to see if it holds true for that specific patient.

```
# Assuming 'm' is the model returned by solver.model()
# and 'pneumonia_rule' is your Z3 boolean expression for the DMN row

result = m.evaluate(pneumonia_rule, model_completion=True)

if z3.is_true(result):
    print("Patient triggers the pneumonia rule.")
else:
```

```
    print("Patient does not trigger the rule.")
```

Setting model_completion=True ensures that if a variable isn't explicitly defined in the model (e.g., an optional question that wasn't required for the current path), Z3 will safely assign it a default concrete value to complete the evaluation.

2. Using substitute() and simplify() (If loading from a JSON/Dict)

If you have a synthetic patient stored as a standard Python dictionary (e.g., loaded from synthetic_patients.json), you can substitute the symbolic variables in your DMN rule with the concrete Z3 constant values, and then simplify the expression to get a True/False result.

Important: You cannot pass raw Python integers or booleans into the substitute function; you must wrap them in Z3 value types like BoolVal, IntVal, or RealVal.

```
from z3 import *

# Define your symbols
cough = Bool('cough')
temp_c = Real('temp_c')
pneumonia_rule = And(cough, temp_c > 37.5)

# Your patient data from JSON
patient_data = {
    cough: BoolVal(True),
    temp_c: RealVal(38.0)
}

# 1. Substitute variables with concrete values
concrete_expr = substitute(pneumonia_rule, *patient_data.items())

# 2. Simplify down to a final Boolean
result = simplify(concrete_expr)

print(result) # Output: True
```

3. Using a New Solver Instance (Assertion Checking)

Another highly reliable way to run a predefined patient through the logic is to spin up a fresh solver, assert the patient's exact data as hard constraints, and then ask the solver if a specific rule is satisfiable under those conditions.

```
s = Solver()

# Lock in the patient's exact state
s.add(cough == True)
s.add(temp_c == 38.0)

# Check if the rule is true for this patient
s.add(pneumonia_rule)
```

```
if s.check() == sat:
    print("Rule fires for this patient.")
else:
    print("Rule is skipped.")
```

Best Practice for Your Pipeline
If you are doing differential testing (comparing Z3's DMN execution against the ODK XLSForm execution), Approach #2 or #3 is usually best. You iterate over your synthetic_patients.json, translate the JSON values into Z3 Val objects, run them through the compiled Z3 DMN rules to record the expected "Next Module" or "Diagnosis", and then feed that exact same JSON into your headless ODK robot to ensure the outputs match perfectly.

Red team critique of the above

This is a highly ambitious and sophisticated approach. Applying formal verification (SMT/Z3) to clinical decision support is the gold standard for safety.

However, as a Red Team auditor, I must attack the abstraction gap. The fundamental vulnerability in your plan is the assumption that the mathematical universe of Z3 maps 1:1 with the runtime execution environment of Open Data Kit (ODK) / XPath. It does not.

If you use this Z3 plan as written, your pipeline will mathematically "prove" that a form is safe, generate a green light, and then silently fail or drop patients in the field because the runtime engine violates Z3's mathematical axioms.

Here is the Red Team teardown of your Z3 verification plan, categorized by the "silent killers" it misses.

1. The Soundness Bug: Z3 Reals vs. XPath Floats/NaN

In your intern guide, you instruct them to map variables like temp_c to Z3 Real and define thresholds like has_fever <-> temp_c > 37.7.

The Trap: In the Z3 theory of Reals, numbers are continuous, infinite-precision, and completely ordered. In Z3, the statement (x > 37.7) \lor (x \le 37.7) is an absolute tautology. It is always true.

However, ODK evaluates logic using XPath 1.0, which utilizes IEEE 754 floating-point math. If a Community Health Worker skips the temperature question, ODK sets it to an empty string. If that is cast to a number for comparison, it evaluates to NaN (Not a Number). In IEEE 754 and XPath, NaN > 37.7 is False, and NaN <= 37.7 is also False.

The Impact: Z3 will prove your table is "Collectively Exhaustive" because it assumes every patient must either have a fever or not have a fever. In reality, a missing temperature creates a state where neither rule fires, the patient falls into a gap, and the ODK form reaches a dead end. Your formal proof will be unsound.

The Fix: You cannot use raw Z3 Real or Int without modeling missingness. You must implement a 3-valued logic system or explicitly model NaN. Every numeric variable must be a Z3 Datatype or a Tuple: (is_present: Bool, value: Real).

Note:

- We can use z3 type FloatingPoint that mimics IEEE754 standards.
- We can have a separate dummy for “temp_c is present”

2. The BMC Illusion: Bounded Model Checking for Routers

Your Level 2 Router check utilizes Bounded Model Checking (BMC) to search for loops up to a depth of k (e.g., k=8). If Z3 returns UNSAT, you claim "strong evidence of termination."

The Trap: BMC is an under-approximation. It only proves that there are no infinite loops that happen to repeat within k steps. It does not prove the absence of loops. In complex Community Health Toolkit (CHT) workflows, a patient might navigate through Intake \rightarrow Vitals \rightarrow Fever \rightarrow Malaria RDT \rightarrow Diarrhea \rightarrow Malnutrition \rightarrow Follow-up. If a state variable is incorrectly cleared at step 7, they could be routed back to Vitals at step 9. Your k=8 BMC will miss this and certify the router as loop-free.

The Fix: To prove true termination, you must implement k-induction or use a solver algorithm like IC3/Property Directed Reachability (PDR). Alternatively, you must strictly constrain the router architecture to be a Directed Acyclic Graph (DAG) and use a topological sort rather than relying on bounded state-space exploration.

3. The DOMAIN Axiom Vulnerability

Your architecture relies heavily on the DOMAIN constraint (e.g., DOMAIN $\wedge$ r.when).

The Trap: The entire validity of your proof rests on the manual authoring of the DOMAIN. If the domain is over-constrained (e.g., someone writes And(age_months >= 0, age_months <= 60) but the form allows entering 65 months), Z3 will ignore rule gaps for 65-month-olds, reporting no failures. If the DOMAIN fails to capture biological mutually exclusive states (e.g., failing to constrain that v_muac_color cannot simultaneously be "Red" and "Green"), Z3 will flood your report with impossible "Overlap" failures.

The Fix: DOMAIN constraints cannot be hand-written. They must be mechanically derived from the Canonical_DataDictionary.csv and predicates.json schemas. If a variable is a select_one in ODK, the compiler must automatically generate exactly-one Z3 constraints (e.g., using Z3 algebraic Datatypes instead of Strings).

4. Feature Engineering Disconnect

Your diagram states: Raw Data -> Feature Compiler -> Boolean Layer -> DMN.

The Trap: You are proving the correctness of the DMN assuming the Feature Compiler perfectly matches the ODK runtime calculates. If your Z3 model defines has_fever == (temp_c > 37.5) but the compiled XLSForm calculates it as if(${temp_c} >= 37.5, 1, 0), you have an off-by-one boundary bug at exactly 37.5. The Z3 proof will pass, but the app will behave differently.

The Fix: The Z3 constraints must not be written separately. You must write a parser that takes the exact XPath strings from your predicates.json (or XLSForm calculates) and mechanically translates those specific strings into Z3 Abstract Syntax Trees. You must verify the actual code that will run, not a parallel Python representation of it.

5. Boundary Triplet Generation Flaw

For synthetic patients, your prompt generates threshold tests: X - 0.1, X, X + 0.1.

The Trap: Floating-point representations cannot accurately hold all base-10 decimals. Testing 37.6 might actually test 37.599999999999994 in the runtime engine. Depending on how the threshold is written, precision errors could cause false positives or negatives in testing.

The Fix: Do not ask Z3 to simply solve for X - 0.1. You must ask Z3 to find the maximum satisfying assignment strictly less than X, and the minimum satisfying assignment strictly greater than X, constrained by the precision limits of the device (e.g., thermometers usually only have 1 decimal place of precision).

Summary

The Z3 integration is a brilliant safeguard, but you are currently verifying an idealized mathematical model, not the hostile environment of a mobile data collection tool. To make this production-ready for clinical use:

* Wrap all Z3 Real/Int variables in a missingness-aware structure (handling null/NaN).

* Mechanically derive Z3 constraints directly from the ODK XPath expressions.

* Replace Bounded Model Checking for the router with k-induction, or statically enforce a DAG.

Note: DMN can use Booleans and Z3 can use native type BOOL. But XLSForm should probably use integers 1 and 0 for TRUE and FALSE so missing values are handled gracefully